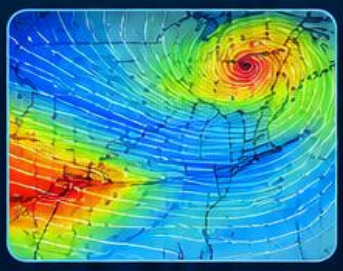
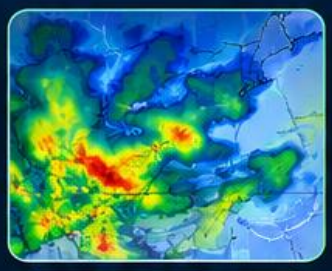
wrfall_install.pl

Install all versions and types of WRF using a single-line command

```
(base) goitomk@gjob:~/$ ./wrfall_install.pl --install list
   *  Finding a perfect home - Location, Location, ...; then Simulation
   Where would you like to install WRF? [/home/goitomk/Build_WRF]:
   Welcome to WRFall. You are installing WRF/WPS.
   *  Installed release : 4.7.0
   *  Summary of available WRF/WPS releases
        V4.8.0    <- Most current
        V4.7.1
        V4.7.0    <- Currently installed
        V4.6.1
        V4.6.0
        .
        .
        .
        V4.0
(base) goitomk@gjob:~/$ ./wrfall_install.pl --install 4.8.0 -all
Insstalling WRF-ARW · WRF-Chem • WRF-Hydro • WRF-Fire ...
Downloading sources ...
Configuring ...
Building ...
Installing ...
Done. ✅
```



WRF-ARW



WRF-Chem



WRF-Hydro



WRF-Fire

Better Model - Better forecast - A safer future

Prepared Goitom Kelem | goikelem@gmail.com | October 2026

Organised by AI

About This Manual

This manual is the complete user guide for **wrfall_install.pl**, a self-contained Perl installer and operations tool for the Weather Research and Forecasting (WRF) modeling system and its companion preprocessor, the WRF Preprocessing System (WPS). It explains everything the tool can do, from the first smoke test on a bare login node to the daily workflow of creating domains, downloading meteorological data, and running end-to-end forecasts. Every flag, default value, directory, and message documented here was verified against version 1.4.2 of the script, which is a faithful Perl port of the original Python tool of the same name.

The manual is written for three audiences. **New users** should read Chapters 1 through 4 in order; they cover what the tool is, what it needs, and how to go from an empty account to a compiled WRF and WPS stack with one command. **Day-to-day operators** will live in Chapters 9 through 11, which document domain creation, data download, and the run pipeline. **System administrators and power users** will find the exhaustive command-line reference in Chapter 5, the library build internals in Chapters 7 and 8, and the troubleshooting playbooks in Chapter 13. The appendices collect reference tables that are worth printing and keeping next to the terminal.

Several passages quote the terminal verbatim, and three of them are full annotated transcripts captured on the reference machine - an 8-core Intel Core Ultra 7 laptop running Ubuntu 24.04 LTS under WSL2, with the Perl script kept in `/home/user` and the installation home at `~/Build_WRF`. Section 4.7 shows the `--install listall` catalog, Section 4.8 the full `--install 4.7.0 -all` session, and Section 9.1 the creation of the example domain **ethiowrf**; the reference values that appear in them (user name `goitomk`, host `gjob`) will, of course, be yours when you run the tool.

Chapters are numbered continuously, and cross-references use the form “see Chapter 5.2”. Commands you type at the shell are shown in monospaced boxes; placeholders you must replace appear in angle brackets, for example <domain>. The tool itself has a playful personality - its greetings, warnings, and fatal messages are deliberately informal - and this manual quotes some of them verbatim so you can recognize them in your terminal.

Convention	Meaning
`wrfall_install.pl --install -arw`	A command to type at the shell prompt, exactly as shown.
<placeholder>	A value you must substitute, such as a directory or a date.

Convention	Meaning
Bold	A key term, a button, or a decision point you should not skip.
“Quoted text”	Verbatim output produced by the tool itself.
Note blocks	Practical advice, common pitfalls, and clarifications.

Related documentation that this manual does not replace: the **WRF Users Guide** (namelist reference and physics selection), the **WPS User Guide** (preprocessing details), the UCAR/NCAR **online compile tutorial** (background on the libraries), and the release notes of each WRF version on GitHub. When this tool wraps one of your own shell scripts, such as `setup_wrf_domain.sh` or `run_wrf.sh`, the corresponding chapter names the script and explains where it is stored under your installation home.

Two safety notes before you begin. First, the tool never needs root: everything is installed under a directory you own, and it will refuse nothing but will happily fill a small disk - plan at least 40 GB for a basic stack with the mandatory geographic dataset, and more for chemistry or fire data. Second, the tool clears proxy environment variables at startup; if your site requires a proxy, pass it with `--proxy HOST:PORT` on every invocation rather than relying on the login-shell environment.

Table of Contents

About This Manual I

1. Introduction 1

1.1 What Is wrfall_install.pl? 1
1.2 Design Goals and Key Features 1
1.3 Supported Build Types 2
1.4 The Wrapped Automation Scripts 2
1.5 Execution Flow at a Glance 3
1.6 Version History 4

2. System Requirements 5

2.1 Operating System and Perl 5
2.2 Hardware Planning 5
2.3 Compilers and System Tools 6
2.4 Network Reachability 7
2.5 Privileges and Account Setup 7
2.6 Sizing a Small Teaching Cluster 7

3. Preparing the Tool 9

3.1 Obtaining the Script 9
3.2 Verifying the Environment 9
3.3 Command Syntax and Help System 10
3.4 First Smoke Tests 10

4. Quick Start Walkthrough 11

4.1 The One-Command Install 11
4.2 What Happens, Stage by Stage 11
4.3 Reading the Console and the Logs 12
4.4 Post-Install Checklist 12
4.5 A Full Cycle in Twelve Commands 12
4.6 A Second Walkthrough: The Chemistry Stack 13
4.7 Annotated Transcript: Listing Everything with --install listall 13
4.8 Annotated Transcript: The Full --install 4.7.0 -all Session 15

5. Command-Line Reference 20

5.1 Primary Commands 20
5.2 Build-Type Selectors 21
5.3 Install Control Flags 21
5.4 Directories and Downloads 22
5.5 Compiler Overrides 22

5.6 Data and Run Flags 23
5.7 Automation and Housekeeping 23
5.8 Exit Codes 24
5.9 Environment Variables 24

6. Installation Layout 25
6.1 How the Home Directory Is Chosen 25
6.2 Directory Map 25
6.3 Anatomy of a Run Domain 26
6.4 Release Coexistence and Scouring 27
6.5 Generated Environment Files 27

7. Building the Library Stack 28
7.1 The Nine Libraries 28
7.2 Build Order and Why It Matters 28
7.3 The Parallel Download Engine 29
7.4 Local Repositories and Package Reuse 29
7.5 Rebuild Strategies 29

8. Compiling WRF and WPS 30
8.1 Configure Choices Made for You 30
8.2 Build Types in Depth 30
8.3 Chemistry Builds: flex, byacc, and KPP 30
8.4 gfortran 10 and Newer 31
8.5 What You End Up With 31

9. Creating Run Domains 32
9.1 The --mkdomain Workflow 32
9.2 Fire Domains 34
9.3 Editing the Domain 34
9.4 Importing and Refreshing Domains 34
9.5 The Wizard and the Utility Scripts 35
9.6 Naming and Organizing Multiple Domains 35

10. Downloading Meteorological Data 36
10.1 The Three Sources 36
10.2 Choosing the Area 36
10.3 Controlling Time 36
10.4 ERA5 Prerequisites 37
10.5 Storage Layout and Reuse 37
10.6 Verifying What You Downloaded 37

11. Running WRF End to End 39

11.1 The --run Command 39
11.2 The Pipeline, Stage by Stage 39
11.3 Monitoring a Run 40
11.4 Rank Counts and Hyper-threading 40
11.5 Running Fire Cases 40
11.6 Interrupted Runs and Reruns 40

12. Updates and Maintenance 42
12.1 Updating to a Newer Release 42
12.2 Multiple Build Types Side by Side 42
12.3 Housekeeping 42
12.4 Moving or Removing an Installation 42

13. Troubleshooting and FAQ 44
13.1 A General Debugging Strategy 44
13.2 Common Failures and Fixes 44
13.3 Frequently Asked Questions 45
13.4 Getting Further Help 46

Appendix A. Complete Flag Reference 47

Appendix B. Directory and File Manifest 49

Appendix C. Glossary 51

Appendix D. Document Information 53

Appendix E. Quick Reference Card 54

Appendix F. WRF / WPS Domain Generator & Wizard — User Guide 55
Contents at a glance 55
What the program does 56
How to read the screenshots 56
Before you start 57
STEP 1 Open the application and learn the layout 57
STEP 2 Start a project 58
STEP 3 Choose projection, domain count and timing 59
STEP 4 Enter domain extents and resolution 61
STEP 5 Enter output and geography data paths 62
STEP 6 Switch on optional model components 63
STEP 7 Edit the component parameters 64
7.1 WRF-Chem – the &chem panel 64
7.2 WRF-Fire – the &fire panel 65
7.3 WRF-Hydro – hydro.namelist 66
7.4 HRLDAS / Noah-MP – namelist.hrldas 67

7.5 Extra physics and generate buttons 69
STEP 8 Use the Interactive Domain Wizard 69
8.1 The toolbar 69
8.2 Select, move and resize 70
8.3 Right-click menu 72
8.4 Draw a domain with the mouse 72
STEP 9 Generate and validate the namelists 73
Search inside the files 75
STEP 10 Download the files and run WRF 76
Dark theme 76
Appendix A Worked example 77
Appendix B Parameter reference 77
&chem 78
&fire 78
hydro.namelist (&HYDRO_nlist) 79
namelist.hrldas – paths 79
Appendix C Troubleshooting 79

Appendix G. WRFall User Guide: Downloading Data and Running WRF 81

1. What WRFall does 81
2. Before you start 81
2.1 Work from inside your domain 81
2.2 Domain folder layout 81
2.3 Check namelist.wps first 82
3. Downloading data (--getdata) 82
3.1 Basic command 82
3.2 What you should see 82
3.3 Options 83
3.4 Examples 84
3.5 Good to know 84
4. Running WRF (--run) 84
4.1 Basic command 84
4.2 What happens, step by step 85
4.3 What you should see 86
4.4 Settings you can change 86
4.5 Settings that stay in your namelists 86
5. Complete daily workflow 87
6. Checking results 87
7. Troubleshooting 87
8. Quick reference 88

1. Introduction

1.1 What Is wrfall_install.pl?

wrfall_install.pl is a single-file Perl program that turns a plain Linux account into a working WRF modeling environment without ever asking for root privileges. One invocation downloads, compiles, and installs the complete toolchain: the MPICH message-passing library, the netCDF data libraries and their HDF5 foundation, the compression libraries that WPS needs to read GRIB files, and finally WRF and WPS themselves. After the build it can create ready-to-run domain directories, fetch initial and boundary data from GFS, ECMWF IFS, or ERA5, and drive the whole forecast pipeline from `link_grib` to `wrf.exe` with a single command.

The script is a full one-to-one Perl port of the Python tool `wrfall_install.py` version 1.4.2, covering its more than seventy internal functions and its exact command-line semantics. Both editions are distributed together through the project's GitHub repository (`github.com/goikelem/WRFall`): the repository ships the original Python file, and the Perl edition documented in this manual accompanies it as `wrfall_install.pl`. The tool requires only Perl 5.14 or newer and modules from the Perl core, so it runs unchanged on minimal servers, older workstation distributions, HPC login nodes where users cannot install software system-wide, and Windows machines running WSL. Everything it builds lives inside one directory tree that you own, which makes updates, side-by-side versions, and clean removals painless.

1.2 Design Goals and Key Features

The tool follows the structure of the classic install_wrf shell scripts but automates every interactive decision, and it adds the operational pieces those scripts never covered. Its design can be summarized in six goals, each of which is visible in the flag set documented in Chapter 5.

- **Zero root, zero system pollution.** Compilers aside, nothing is written outside `<home>`; the tool even generates the `modulefile` entries so your site `environment modules` can load the stack.
- **One command, full stack.** `--install -arw` goes from an empty directory to compiled WRF and WPS binaries, including the geographic base data.
- **Resumable and parallel downloads.** Large tarballs are fetched in parallel HTTP range segments (eight workers by default) with per-segment resume markers, so a dropped connection after an hour of downloading costs minutes, not hours.

- **Side-by-side releases.** A newer WRF can be built next to the current one, reusing the already-compiled support libraries, and old releases can be scoured afterwards.
- **Operational workflow included.** Domains (`--mkdomain`), data (`--getdata`), and runs (`--run`) are first-class commands, ported from daily-use automation scripts.
- **Predictable automation.** `--allyes`, `--dryrun`, and argparse-compatible error messages make the tool usable from shell scripts and CI pipelines.

1.3 Supported Build Types

A build type selects which WRF variant is compiled and how WPS is prepared. The type is chosen with short flags such as `-chem` or the long form `--type chem`, and several types may be requested on one command line. The `-fire` type deserves a special mention: it builds the standard real-data WRF binary but additionally prepares WPS for the fire grid, an arrangement described in detail in Chapter 8.3 and Chapter 9.2.

Table 1. Build types and what each one produces

Type	Flag	WRF compile target	What you get
ARW	-arw	em_real	The standard advanced-research real-data stack with WPS.
Chem	-chem	em_real with KPP	WRF-Chem with local flex, byacc, and KPP support; extra terrestrial chemistry datasets offered.
Hydro	-hydro	em_real + Hydro	WRF-Hydro v5.3.0 coupling layer from NCAR built alongside ARW.
Fire	-fire	em_real	Real-data WRF-Fire: WPS gets NFUEL_CAT and ZSF fire entries, high-resolution fire datasets are offered.
FireIdeal	-fireideal	em_fire	Idealized fire cases such as hill_simple and two_fires; WPS is not built.
All	-all	all of the above	Installs arw, chem, hydro, and fire in sequence (fireideal is opt-in only).

1.4 The Wrapped Automation Scripts

The installer grew out of a set of shell and Python scripts that its author used for daily operations. Those scripts are still shipped - the `--addpack scripts` package installs them under `<home>/Scripts` - and each high-level flag of the Perl tool is a direct

descendant of one of them. Knowing the lineage helps when you read older documentation or compare behavior.

Table 2. Original scripts and the flags that superseded them

Original script	Superseded by	Purpose
install_wrf_4.6.0.sh	--install / --update	Libraries, WRF and WPS builds, environment files, and the post-install Do-List.
setup_wrf_domain.sh	--mkdomain	Domain directory creation with symlinks and namelist preparation.
download_025_gfs.sh	--getdata gfs	GFS 0.25-degree initial and boundary condition downloads.
download_ecmwf_ifs_025.sh	--getdata ecmwf	ECMWF IFS 0.25-degree open-data downloads.
download_era5.sh / .py	--getdata era5	ERA5 reanalysis downloads through the Python cdsapi client.
run_wrf.sh	--run	End-to-end ungrib, metgrid, real, and wrf execution pipeline.

1.5 Execution Flow at a Glance

Every invocation follows the same eight-stage skeleton, which mirrors the structure of the original Python tool. Understanding the skeleton makes log files readable, because each stage announces itself with a banner line and writes its own log under `<home>/logs`. The stages are: initialize, parse flags, greet, configure, list packages (when you asked for a listing), the main action - install, update, or addpacks - then post-configure and finally guidance, where the tool prints its closing advice and the famous Do-List.

Two properties of the flow are worth remembering. First, configuration is cumulative but overridable: the tool derives defaults from the environment (for example `$WRF_HOME`), from the recorded metadata of a domain you may be standing in, and finally from its built-in defaults, with command-line flags winning over everything. Second, the main action is idempotent by design - running `--install` again with `--continue` skips libraries that are already present, and `--update` detects the previous release automatically.

1.6 Version History

The Perl edition carries the full change history of the Python original, because behavior is identical. The table below condenses the release notes that are also embedded in the script header. When reporting an issue, always quote the version printed by `--version`.

Table 3. Change log condensed from the script header

Version	Highlights
1.2.0	Parallel HTTP range downloads (`--dl-jobs`, default 8), per-segment resume after Ctrl-C, streamed GRIB downloads, live throughput display.
1.3.0	Domain sub-directories per cycle (GFS_Input/YYYYMMDDCC), wrf_output layout, shared Scripts folder, `--domain a,b,c`, `--run auto`, wrfall soft link.
1.4.0	wrf_output/{metgrid,ungrib,Plots,geogrid}, one shared Scripts home, `--getdata` reads the d01 extent from namelist.wps, context-aware messages.
1.4.1	Chem builds: absolute YACC path, pre-build yacc/flex/libfl check, KPP failure hint.
1.4.2	Chem builds: FLEX environment export, configure.wrf YACC/FLEX re-pointing, flex/yacc smoke test, WRF 4.7 KPP bug warning, richer KPP diagnostics.

2. System Requirements

2.1 Operating System and Perl

The tool targets 64-bit Linux - the same platforms WRF itself supports - and has been exercised on Ubuntu, Debian, Rocky/Alma, CentOS, and OpenSUSE families. Perl must be version 5.14 or newer, which every distribution released in the last decade provides by default. Only core modules are required: Cwd, File::Basename, File::Copy, File::Path, List::Util, POSIX, File::Find, Fcntl, Sys::Hostname, Time::HiRes, and Time::Local.

Three optional modules improve the experience but are not hard requirements. `HTTP::Tiny` (core since Perl 5.14) enables the built-in pure-Perl downloader used as the last fallback; for HTTPS URLs it additionally needs `IO::Socket::SSL`, and without it the tool simply prefers wget or curl. `JSON::PP` (core since 5.14) is used to read the release list from the GitHub API and to write domain metadata. If either module is missing the tool degrades gracefully and says so during startup. Python 3 is needed only for ERA5 downloads, where the official `cdsapi` client is invoked; the `--addpack cdsapi` command installs it into your user site-packages with pip.

You can verify your interpreter in one line before doing anything else. The command below prints the Perl version and confirms that the two most important optional modules load. A response ending in two `ok` lines means the environment is fully capable; missing SSL support only means wget or curl will be preferred for downloads.

```
perl -v | head -2

perl -MHTTP::Tiny -e 'print "HTTP::Tiny ok\n"'    # optional, used by
fallback fetcher

perl -MJSON::PP   -e 'print "JSON::PP ok\n"'      # optional, release
list + domain metadata
```

2.2 Hardware Planning

WRF compilation is CPU- and memory-hungry, and the geographic datasets are genuinely large. The installer shows planning figures during the install interview so you can abort before a disk fills up, but it is wiser to size the machine up front. The figures below are the ones the tool itself quotes; treat them as floors, not ceilings, because forecast output grows quickly with domain size and run length.

Table 4. Disk space planning figures used by the installer

Component	Planning size	Notes
Support libraries	2 GB	zlib, HDF5, netCDF-C/Fortran, MPICH, libpng, Jasper, flex, byacc under `<home>/LIBRARIES` and `<home>/local`.
WRF build	3 GB	Source tree plus objects and executables per build type.
WPS build	1 GB	Source tree plus geogrid, ungrib, and metgrid executables.
WPS_GEOG base	30 GB	Mandatory high-resolution terrestrial dataset downloaded by default.
Chemistry geog data	12 GB	Eight extra datasets (MODIS land use, soil types, albedo, erodibility, clay and sand fractions).
Working space	+20 GB	Per domain: GRIB input for one cycle plus wrf_output for a 72-hour run.

For memory and CPU, 8 GB of RAM is the practical minimum for compiling WRF, and 16 GB keeps parallel `make -j` builds comfortable on an 8-core machine. More cores shorten the compile (which takes roughly 20 to 60 minutes depending on the type) but do nothing for download speed, which is governed by `--dl-jobs` and your bandwidth instead. A forecast run itself is domain-dependent; the tool warns you at startup if it detects hyper-threading, because one MPI rank per physical core is almost always faster.

When you request several build types at once, the tool prints its own combined planning figure during the install interview. The arithmetic behind it is transparent: 2 GB of libraries, plus 4 GB (WRF plus WPS) for each requested type, plus 30 GB for the mandatory geographic dataset, plus 12 GB more when Chemistry is among the types. A full `--install 4.7.0 -all` install therefore quotes “about 60 GB (planning figure; check the dataset sizes you actually select)” - the annotated transcript in Chapter 4.8 shows the line in context. Machines that skip the geographic data with `--nogeog` drop 30 to 42 GB from the estimate at a stroke.

2.3 Compilers and System Tools

The build assumes the GNU toolchain: `gcc`, `g++`, `gfortran`, `make`, and standard utilities such as `tar`, `gzip`, and `patch`. Any gfortran from 7 onward works; on version 10 or newer the tool automatically adds `-fallow-argument-mismatch` where WRF-Chem needs it (see Chapter 8.4). A download client is required - `wget` is preferred, then `curl`, then the built-in Perl fetcher - and `git` is needed only for the Domain Wizard package. All of these

are installable by your administrator once; nothing is ever installed into system paths by this tool.

```
# Debian/Ubuntu family
sudo apt install build-essential gfortran perl wget curl git csh libio-socket-ssl-perl libnet-ssleay-perl


# RHEL / Rocky / Alma family
sudo dnf install gcc gcc-c++ gcc-gfortran make wget curl git tcsh perl-IO-Socket-SSL perl-Net-SSLeay
```

2.4 Network Reachability

Downloads come from a handful of well-known hosts: GitHub for WRF, WPS, netCDF, zlib, flex, and the Wizard repository; the HDF Group for HDF5; mpich.org for MPICH; SourceForge for libpng; and the UCAR MMM site for Jasper and the WPS_GEOG datasets. Data downloads hit NOAA's NOMADS for GFS, the ECMWF open-data server, and the Copernicus CDS for ERA5. If your site blocks direct egress, point the tool at your proxy with `--proxy HOST:PORT`, or stage all tarballs in a directory and pass `--repodir DIR` - the installer treats a local repository exactly like the network, including checksum-friendly resume behavior.

2.5 Privileges and Account Setup

Nothing in the workflow requires root. The one genuine prerequisite is a home (or work) filesystem with tens of gigabytes of quota where you can create the installation directory. Avoid NFS mounts that do not support POSIX locks if you plan parallel downloads; local or Lustre/GPFS scratch is ideal. If you install on a cluster login node, remember that compile jobs should ideally run on a compute node or be limited with the `--jobs` style resource controls of your scheduler; the tool itself simply runs `make` in the background and logs everything under `<home>/logs` so an interrupted session can be diagnosed afterwards.

Tip. Set `export WRF_HOME=~/Build_WRF` in your `~/.bashrc` before the first install. The tool uses it as the default installation root, and every later command - update, mkdomain, getdata, run - picks it up automatically.

2.6 Sizing a Small Teaching Cluster

Classrooms and small research groups often run the installer once on a shared filesystem and let many accounts use the result. The layout supports this directly: give the group a project directory as `<home>`, install there once, and point every student's `WRF_HOME` at it. Domains are personal - each student creates their own under the shared home, which keeps namelists,

data, and outputs from colliding - while the release directories, libraries, and WPS_GEOG are read-only shared assets used by everyone.

Plan roughly 45 GB for the shared side (libraries, one ARW release, WPS_GEOG) plus 20 to 30 GB per active student domain per case. For the shared-download concern, remember the tarfile cache: the first student to trigger a download pays for it, and everyone after that reuses `<home>/packages` for free. If your cluster schedules jobs, compile WRF on a compute node or an interactive allocation; the build itself honors nothing but `make`, so a 30-minute interactive slot on any 8-core node is enough for the ARW type.

3. Preparing the Tool

3.1 Obtaining the Script

The official distribution channel is the project's GitHub repository. Cloning it gives you the tool together with its documentation in one step, and it makes staying current a one-command operation later (`git pull`). The repository ships the original Python edition, `wrfall_install.py`; the Perl edition documented in this manual, `wrfall_install.pl`, accompanies it and is functionally identical - every flag, default, and console message described here applies to both. The standard first steps are:

```
git clone https://github.com/goikelem/WRFall.git
cd WRFall
chmod +x wrfall_install.py        # make the Python edition executable
./wrfall_install.py --version       # or:  perl wrfall_install.pl --
version
```

Whichever edition you adopt, keep the script outside the installation home. `<home>` may be scoured or rebuilt between releases, and you do not want the installer deleted along with an old stack. On the reference machine quoted throughout this manual, the Perl script lives in `/home/user/wrfall_install.pl`, comfortably outside `~/Build_WRF`, and every domain's `wrfall` link records exactly that absolute path (Chapter 9.1 shows it in the create-domain output). A conventional alternative is `~/bin`, with the executable bit set:

```
cp wrfall_install.pl ~/bin/          # or any directory on your PATH
chmod +x ~/bin/wrfall_install.pl
~/bin/wrfall_install.pl --version    # prints: wrfall_install.pl 1.4.2
```

Because the file is self-contained, upgrading the tool is just replacing the file - or running `git pull` in the clone and copying the new version into place. The version number in the header comment and the `--version` output are authoritative. If you maintain several machines, ship the same file everywhere; all state lives under each machine's `<home>`, never inside the script.

3.2 Verifying the Environment

Two one-liners from Chapter 2.1 confirm Perl and its optional modules. Beyond that, the tool checks its own surroundings at startup: it verifies that the compilers exist, that the download client works, and that the target directory is writable. You can watch all of it happen without side effects by adding `--dryrun` to any command - nothing is downloaded, nothing is compiled, and every action that would have been taken is printed instead. This is

the safest way to preview behavior on a new cluster before committing tens of gigabytes of downloads.

The one environment variable the tool honors by default is `WRF_HOME`. When unset, and when you are not standing inside a domain directory, the install interview asks where to build and offers `~/Build_WRF` as the default answer. When set, every command assumes it, which is what most operators want after day one. The proxy variables `http_proxy` and `https_proxy`, by contrast, are deliberately cleared at startup; pass `--proxy HOST:PORT` explicitly if your network needs one.

3.3 Command Syntax and Help System

The command grammar is: one primary flag, optionally a build type, plus any additional options. The parser is argparse-compatible - options may appear in any order, values may be attached with = or given as the next token, and `--` ends option processing. Usage errors produce messages in the Python argparse style, for example “argument --dl-jobs: invalid int value: 'x'”, and exit with status 2. Three housekeeping flags complete the picture: `--help` for the quick-start sheet, `--flags` for the complete option table, and `--version` for the release number.

```
wrfall_install.pl --help       # short usage sheet with examples
wrfall_install.pl --flags      # every flag, one line each
wrfall_install.pl --version    # release identification
```

3.4 First Smoke Tests

Before the big install, run the three cheapest commands. First, list the releases the tool can install - this exercises the GitHub release API (with a built-in fallback list when the API is unreachable) and proves networking works. Second, request the geographic dataset catalog with `--install listgeog` to see the names `--addgeog` accepts. Third, perform a full dry run of the installation to preview every stage on your machine.

```
wrfall_install.pl --install list           # which WRF releases can I
install?
wrfall_install.pl --install listgeog      # which terrestrial datasets
exist?
wrfall_install.pl --install -arw --dryrun # preview the whole install
```

Tip. `--install list` prints a table of the last forty releases. The default release is 4.6.0; a specific version is selected with `--install 4.5`, and the tool refuses to move backwards to an older release unless you add `--force`. The richest of the three listing modes is `--install listall`, which adds the terrestrial dataset catalog and the optional package list; Section 4.7 quotes its complete output and explains every line.

4. Quick Start Walkthrough

4.1 The One-Command Install

With requirements satisfied, the canonical first install is:

```
wrfall_install.pl --install -arw          # the standard first install
wrfall_install.pl --install 4.7.0 -all      # every build type, named
release
```

If `WRF_HOME` is unset the tool opens with its interview - "Finding a perfect home - Location, Location, ..., then Simulation" - and asks where to install, defaulting to `~/Build_WRF`. It then greets you, plans the disk budget (2 GB libraries, 3 GB WRF, 1 GB WPS, 30 GB geographic data), and starts the download engine. All questions accept the default by pressing Enter; `--allyes` suppresses them entirely for scripted runs. The second form above is what the reference machine used for the annotated transcript in Section 4.8: `-all` expands to ARW, Chem, Hydro, and Fire, and the named release 4.7.0 is pinned instead of the default 4.6.0.

4.2 What Happens, Stage by Stage

Knowing the stages turns a forty-minute build from a black box into a checklist. The order below is exactly what you will see in the console and in `<home>/logs`, and each stage is safe to resume with `--continue` if something interrupts it.

1. **Directory skeleton.** `<home>` is created with `LIBRARIES`, `packages`, `local`, `etc/modulefiles`, and `logs` sub-directories.
2. **Library builds.** zlib, HDF5, netCDF-C, netCDF-Fortran, MPICH, libpng, Jasper, flex, and byacc are downloaded (8 parallel segments each by default), unpacked, configured, compiled, and installed under `<home>/local`.
3. **WRF source.** The release tarball (4.6.0 by default) is fetched from GitHub, unpacked as `WRF-4.6.0-ARW`, configured with option 34 (gfortran, dmpar), and compiled - the longest stage.
4. **WPS source.** The matching WPS is configured against the just-built WRF and compiled into `WPS-4.6.0-ARW`.
5. **Geographic data.** The mandatory WPS_GEOG dataset is downloaded unless `--nogeog` was given; a Chemistry install also offers the eight chemistry datasets.
6. **Environment files.** The tool writes env templates and a modulefile, then prints the Do-List - the closing guidance you will see every time.

4.3 Reading the Console and the Logs

Console output uses four marker characters: `*` for information, `!` for warnings, a check-mark for success, and a skull for fatal errors. Fatal errors always end with the reassuring pair “Something died in the pursuit of science - Again!” and “Fix the problem above and run me again.” - the actionable message is always the line above them. Every build command's full output is also appended under `<home>/logs`, one file per library and per model, which is where you look when a compile fails and the console only shows the last screen. Pressing Ctrl-C is caught politely (“Ouch! - Aren't we a bit sensitive?!”) and exits with status 2; partial downloads keep their per-segment markers and resume on the next run.

4.4 Post-Install Checklist

When the Do-List appears, the stack is complete. Before your first real run, verify these four points; each takes seconds and prevents the classic first-day surprises.

- `ls ~/Build_WRF` shows `WRF-4.6.0-ARW` and `WPS-4.6.0-ARW` with executables (`wrf.exe`, `real.exe`, `geogrid.exe`, `ungrib.exe`, `metgrid.exe`) present.
- `source <home>/env_wrf.sh` (or the file the Do-List names) puts netCDF and MPICH on your paths - the tool prints the exact line to add to `~/.bashrc`.
- `<home>/WPS_GEOG` contains the geographic dataset; note its path, because `--mkdomain` will ask for it.
- `geogrid.exe` runs in a test domain without errors (Chapter 9 walks through creating one).

4.5 A Full Cycle in Twelve Commands

The sequence below is the complete operator loop on a fresh machine, from an empty account to a finished forecast: obtain the script, install, prepare the shell, create a domain, fetch GFS data for a date and cycle, and run. Chapters 3 and 9 through 11 explain each command in depth; here it is as a worked script you can adapt verbatim, using the example domain name **ethiowrf**.

```
git clone https://github.com/goikelem/WRFall.git && cd WRFall
cp wrfall_install.pl ~/bin/ && chmod +x ~/bin/wrfall_install.pl
wrfall_install.pl --install -arw --allyes
echo 'export WRF_HOME=~/Build_WRF' >> ~/.bashrc && source ~/.bashrc
source ~/Build_WRF/env_wrf.sh
wrfall_install.pl --mkdomain ethiowrf
cd ~/Build_WRF/ethiowrf
./wrfall --getdata gfs --date 20261016 --end 72
./wrfall --run 20261016 00 72
ls wrf_output/2026101600/          # wrfout files land here
tail -f rsl.error.0000             # watch the run progress
./wrfall --update                  # months later: next release
```

Why `./wrfall`? Every domain directory contains a `wrfall` soft link back to the installer. Running it from inside a domain automatically targets that domain and its recorded home, so you never have to type `--domain` again. The same effect works from any directory by passing `--domain ~/Build_WRF/ethiowrf`.

4.6 A Second Walkthrough: The Chemistry Stack

Adding WRF-Chem to an existing ARW machine shows how the tool reuses everything it can. The command `wrfall_install.pl --install -chem` recognizes the healthy library stack, skips all nine libraries without recompiling, fetches the WRF source once more, and builds it with the KPP toolchain: local flex and byacc are compiled first, the FLEX environment variable is exported, and the generated configure.wrf is patched so KPP's generator steps use the local tools. The result is a sibling `WRF-4.6.0-Chem` directory beside `WRF-4.6.0-ARW`.

After the model compiles, the tool offers the eight chemistry terrestrial datasets - about 12 GB - which you can accept interactively or take later with `--addgeog chem`. Accepting immediately is convenient; deferring is equally safe, and the datasets land in the same WPS_GEOG tree either way. Two reminders close the walkthrough: stay on release 4.6.x for Chemistry until upstream resolves the 4.7 KPP build bug (Chapter 8.3), and expect the Chem build to take roughly twice as long as ARW because the mechanism files are generated and compiled per mechanism.

4.7 Annotated Transcript: Listing Everything with --install listall

The fastest way to see what an installation offers is the catalog command. The transcript below was captured on the reference machine after release 4.7.0 had been installed; it is

reproduced exactly as the screen showed it, and the paragraphs that follow explain each block. Running the command is completely safe - it only reads state and prints.

```
$ ./wrfall_install.pl --install listall

    Welcome to WRFall. You are installing WRF/WPS.

    *  Installed release : 4.7.0

    *  Summary of available WRF/WPS releases
        V4.6.0     <- Most current



    *  Terrestrial datasets available
        geog_high_res_mandatory          Base  WPS_GEOG  (installed  by
default)
        modis_landuse_21class_30s    WRF-Chem dataset (group: chem)
        soiltype_top_2m              WRF-Chem dataset (group: chem)
        soiltype_bot_2m              WRF-Chem dataset (group: chem)
        albedo_ncep                  WRF-Chem dataset (group: chem)
        maxsnowalb                   WRF-Chem dataset (group: chem)
        erod                         WRF-Chem dataset (group: chem)
        clayfrac_5m                  WRF-Chem dataset (group: chem)
        sandfrac_5m                  WRF-Chem dataset (group: chem)
        fire                              WRF-Fire folders landfire/ +
highres_elev/ (you supply NFUEL_CAT, ZSF)

    *  Optional packages
        domwiz               WRF   Domain   Wizard   (git   clone   of
https://github.com/goikelem/WRF-domain-Wizard)
        scripts    Your original automation shell/python scripts, saved
under <home>/Scripts
        cdsapi      The  Python  cdsapi  module  (pip  --user)  needed  for
ERA5 downloads

    *  You now have the power - Abuse it wisely!
```

The greeting and the installed-release line. Every invocation opens with a welcome that names the actions you requested - here simply “You are installing WRF/WPS”, because a

listing is a degenerate install. The **Installed release** line reports what the tool found recorded under the installation home; a fresh machine prints "No WRF installed" in its place. This is the quickest way to answer "which release is live here?" on a machine you have not touched in months.

The release summary. The list of installable releases comes from the GitHub release API when the network allows and from a built-in fallback list otherwise. The first entry in that list carries the "<- Most current" marker; the entry matching the installed release would be tagged "<- Currently installed" instead. Note that the marker identifies the head of the tool's list, which can lag the newest release the tool is actually able to install - in this captured session V4.6.0 heads the list while 4.7.0 is both installed and fully installable. Any listed version can be requested by name (`--install 4.7.0`), and the closing line of the transcript changes with the situation: when the installed release equals the head of the list you get "No need for an update at this time", and otherwise the tool signs off with its motto, "You now have the power - Abuse it wisely!".

Terrestrial datasets. The first entry, `geog_high_res_mandatory`, is the base WPS_GEOG set that every install downloads by default (30 GB). The eight entries labelled "group: chem" are the WRF-Chem fields - MODIS land use, the two soil-type layers, albedo, maximum snow albedo, erodibility, and the clay and sand fractions - which `--addgeog chem` fetches as a set (about 12 GB, Chapter 4.6). The `fire` entry is different in kind: rather than downloading data, `--addgeog fire` creates the `landfire/` and `highres_elev/` folders for you to populate with licensed Landfire and elevation assets, because geogrid's NFUEL_CAT and ZSF fields expect them (Chapter 9.2).

Optional packages. The three packages are the ones `--addpack` accepts, each with a one-line description: `domwiz` clones the graphical WRF Domain Wizard from its own GitHub repository, `scripts` installs the wrapped automation scripts under `<home>/Scripts`, and `cdsapi` pip-installs the Python ERA5 client into your user site-packages (Chapter 10.4). The catalog command ends there - nothing was downloaded, compiled, or changed.

4.8 Annotated Transcript: The Full --install 4.7.0 -all Session

This transcript shows the complete opening sequence of the heaviest command the tool offers: installing release 4.7.0 with `-all`, i.e. the ARW, Chem, Hydro, and Fire build types in one session. The capture comes from the reference machine - an 8-core laptop running Ubuntu 24.04 under WSL2 - with an existing library stack, which is why every library step reports a skip. Each block is quoted verbatim and then annotated.

Stage one - the home interview and the plan.

```
$ ./wrfall_install.pl --install 4.7.0 -all

    *  Finding a perfect home - Location, Location, ..., then Simulation
    Where would you like to install WRF? [/home/goitomk/Build_WRF]:

    Welcome to WRFall. You are installing WRF/WPS.

    *  Preparing to install WRF/WPS V4.7.0 for: ARW, Chem, Hydro, Fire

    *  Rough disk space needed: about 60 GB (planning figure; check the
dataset sizes you actually select).
```

The “Finding a perfect home” prompt appears because `WRF_HOME` was unset in this shell; pressing Enter accepts the bracketed default, and setting `export WRF_HOME` beforehand suppresses the question entirely (Chapter 2.5). The welcome line then restates the request, and the preparation line lists the build types after expansion - `-all` became “ARW, Chem, Hydro, Fire”. The disk figure is the transparent arithmetic from Chapter 2.2: 2 GB of libraries, 4 GB per type times four, 30 GB of base geographic data, and 12 GB of chemistry fields - 60 GB in round numbers. It is a planning figure, exactly as the parenthetical says, and the tool compares it against free space on the target filesystem before continuing.

Stage two - the machine survey and the confirmation.

```
*  Processor information dug up for gjob:
  HOSTNAME         : gjob
  SOCKETS          : 1
  CORES PER SOCKET : 8
  TOTAL CORES      : 8


*  System information for gjob:

     Machine Address     : 127.0.1.1
     Linux Kernel        : 6.6.87.2-microsoft-standard-WSL2
     Linux Distribution  : Ubuntu 24.04.5 LTS

     CPU Name            : Intel(R) Core(TM) Ultra 7 268V
     CPU Type            : x86_64
     Sockets             : 1
     Cores per Socket    : 8
     Total Physical CPUs : 8
     Available Memory    : 15.40 GB
Installing WRF 4.7.0. Is this OK by you? [OK by me]:
```

Before committing an hour of compile time, the tool digs up the processor topology - sockets, cores per socket, total cores - and a system panel: kernel, distribution, CPU model and architecture, and available memory. Two things in this example deserve a second look. First, the kernel string `6.6.87.2-microsoft-standard-WSL2` shows the whole stack running on Windows Subsystem for Linux, which the tool treats as an ordinary Ubuntu host - a popular way to run WRF on a Windows laptop. Second, the eight reported cores are the numbers that size the compile parallelism and later the default `WRF_NP` rank count; the startup hyper-threading warning would have appeared here had the tool detected oversubscription. The confirmation prompt ends the stage: Enter accepts the bracketed “OK by me”, and `--allyes` would have answered it automatically.

Stage three - the nine support libraries.

```
    *  Building 9 support libraries into /home/goitomk/Build_WRF/local
      ✓  [1/9] zlib already installed (skipping)
      ✓  [2/9] hdf5 already installed (skipping)
      ✓  [3/9] netcdf_c already installed (skipping)
      ✓  [4/9] netcdf_fortran already installed (skipping)
      ✓  [5/9] mpich already installed (skipping)
      ✓  [6/9] libpng already installed (skipping)
      ✓  [7/9] jasper already installed (skipping)
      ✓  [8/9] flex already installed (skipping)
      ✓  [9/9] byacc already installed (skipping)

    ✓    All  libraries  installed  to  /home/goitomk/Build_WRF/local
(existing ones reused)
```

The library stage is idempotent by design. For each of the nine libraries the tool probes `<home>/local` for the files that prove a working install - headers, shared objects, and configuration binaries such as `nc-config` and `nf-config` - and prints “already installed (skipping)” when the probe succeeds, as it did for every entry on this machine. On a fresh home the same lines read “downloading”, then “configuring”, “compiling”, and “installing” in turn, each writing its log under `<home>/logs`. The closing line confirms the reuse contract: existing, verified libraries are never rebuilt unless you pass `--force-libs`, which is why adding a build type later takes minutes instead of an hour.

Stage four - WRF and WPS, type by type.

```
    *     Downloading   and   compiling   WRF   v4.7.0   (ARW)   -   log:
/home/goitomk/Build_WRF/logs/wrf_arw_4.7.0.log
        WRFV4.7.0.tar.gz                              - Found locally
    *  Compiling WRF 4.7.0 (ARW, ./compile em_real) - this can take 20-
40+ minutes

    ✓  WRF ARW built: /home/goitomk/Build_WRF/WRF-4.7.0-ARW



    *     Downloading   and   compiling   WPS   v4.7.0   (ARW)   -   log:
/home/goitomk/Build_WRF/logs/wps_arw_4.7.0.log
        WPSV4.7.0.tar.gz                              - Found locally

    ✓  WPS ARW built: /home/goitomk/Build_WRF/WPS-4.7.0-ARW

    *  WRF 4.7.0 (Chem) already built (use --force to rebuild)

    *  WPS 4.7.0 (Chem) already built (use --force to rebuild)
```

The model stage walks the requested types in order, ARW first. Each banner names the log file that captures the full download and compile output - `wrf_arw_4.7.0.log` and `wps_arw_4.7.0.log` here - so a failed build is diagnosed from the log, not the console (Chapter 13.1). The “Found locally” line means the tarball was already in the `<home>/packages` cache; a fresh machine shows a download progress display instead. The compile banner sets expectations honestly - 20 to 40+ minutes for ARW on this class of hardware - and the check-mark lines name the completed directories. Because a previous session had already built the Chemistry pair, its WRF and WPS lines report “already built (use --force to rebuild)”; the Hydro and Fire entries continue after them in the same either-build-or-skip pattern, so the session ends with all four type pairs in place, sharing the one library stack.

Note. Transcript line widths vary with terminal size; the wrap points you see on your own screen may differ, but every message above is verbatim output of wrfall_install.pl 1.4.2. Where your home path, user name, or hostname differ, the tool prints yours instead of the reference values shown here.

5. Command-Line Reference

This chapter is the authoritative catalog of every option. Flags are grouped by purpose; Appendix A repeats the complete list in one table for quick printing. Defaults shown here are the values in effect when the flag is omitted, exactly as the internal state of the script defines them.

5.1 Primary Commands

A command line carries at most one primary command; combining `--install` and `--update` is rejected with an explicit error. Commands without a primary flag - for example `--mkdomain` alone - operate on the current installation.

Table 5. Primary commands

Command	Argument	Behavior
`--install`	`[REL]`	Fresh install of WRF/WPS and libraries. The optional release argument accepts a version such as `4.6.0` or a partial like `4.5`, or the keywords `list`, `listall`, `listgeog`. Without an argument the current default release (4.6.0) is installed. The `listall` catalog output is quoted and annotated in Section 4.7, and a full `--install 4.7.0 -all` session in Section 4.8.
`--update`	`[REL]`	Build a newer release next to the current one, reusing compiled libraries, then refresh existing domain directories (unless `--norefresh`).
`--addpack`	`LIST`	Install optional packages, comma-separated: `domwiz` (Domain Wizard), `scripts` (the original automation scripts), `cdsapi` (Python ERA5 client via pip --user). Aliases such as `dwiz`, `utils`, or `era5` are accepted.
`--addgeog`	`LIST`	Install terrestrial datasets: `mandatory`, `chem`, `fire`, or any UCAR dataset name. Repeatable.
`--mkdomain`	`NAME`	Create a run domain under `<home>/NAME`; add `-fire` for a fire-ready domain (Chapter 9).
`--getdata`	`gfs|ecmwf|era5`	Download initial/boundary data for one or more `--domain` directories (Chapter 10).

Command	Argument	Behavior
`--run`	`auto` or `YYYYMMDD CC HH`	Execute the full pipeline - ungrib, metgrid, real, wrf - optionally followed by a source keyword (Chapter 11).

5.2 Build-Type Selectors

Build types were introduced in Table 3. Short flags may be repeated and combined: `-chem -hydro` builds both. The long form `--type` takes the same names (`arw`, `chem`, `hydro`, `fire`, `fireideal`) and is repeatable. `-all` expands to arw, chem, hydro, and fire - fireideal is deliberately excluded because it is a specialist build without WPS. If no type is given anywhere, the install interviews you and defaults to ARW.

5.3 Install Control Flags

These flags shape how the main action behaves: what is skipped, what is rebuilt, and what happens to old releases. They are the ones you will reach for after the first successful install.

Table 6. Install control flags

Flag	Default	Effect
`--continue`	off	Resume mode: skip libraries that are already built and jump straight to the interrupted stage.
`--force`	off	Rebuild WRF/WPS even if present; also permits installing an older release than the current one. Existing support libraries are kept.
`--force-libs`	off	Additionally rebuild the support libraries (zlib, hdf5, netcdf, mpich, flex, byacc, ...).
`--nogeog`	off	Do not download the base WPS_GEOG dataset - useful when it is already on a shared filesystem.
`--import DIR`	none	Import domains (directories containing namelist.wps) from an older installation; repeatable.
`--scour`	off	Delete the previous release directories after a successful install or update.
`--norefresh`	off	With `--update`, leave existing domain directories untouched.
`--scrubpkgs`	off	Delete package tarfiles after installation instead of keeping them under `<home>/packages`.

Flag	Default	Effect
`--nounpack`	off	Download only; no unpacking, no building - a pure staging mode.
`--nolocal`	off	Ignore tarfiles already in `<home>/packages` and download fresh copies.

5.4 Directories and Downloads

Location and transport flags control where things land and how bits move. The download engine selects wget first, then curl, then the built-in Perl fetcher (`HTTP::Tiny`, which needs `IO::Socket::SSL` for HTTPS); `--wget` or `--curl` pins the choice, and specifying both is an error. The user-agent is reported as Wget/1.21.4 because some front-end CDNs - notably the one in front of mpich.org - throttle unknown clients.

Table 7. Directory and download flags

Flag	Default	Effect
`--homedir DIR`	`$WRF_HOME` or `~/Build_WRF`	Top-level installation directory; created if missing.
`--repodir DIR`	none	Take package tarfiles from a local repository directory instead of the network.
`--dl-jobs N`	8	Parallel HTTP range connections per file. `1` selects the single-stream path with classic resume. `--getdata` caps the value at 3 to stay polite to data servers.
`--proxy HOST:PORT`	cleared	Sets `http_proxy`/`https_proxy` for this invocation; proxies are otherwise removed from the environment at startup.
`--curl` / `--wget`	auto	Force one download client.

5.5 Compiler Overrides

The WRF `./configure` menu is answered automatically: choice **34** (x86_64, gfortran, dmpar) for WRF, nesting option **1** (basic), and choice **3** (gfortran, dmpar) for WPS. The three numbers can be overridden when your site standardizes on something else - for example `--wrf-opt 35 --nest-opt 2` for the smpar or dm+sm variants. Overrides

apply to every build type in the invocation, and `--dryrun` shows the menu answers it would pick before anything is compiled.

5.6 Data and Run Flags

The flags in this table parameterize `--getdata` and `--run`. The area of interest is normally derived from the domain's namelist.wps (Chapter 10.2); `--bbox` overrides it explicitly with four numbers in the order **west east south north**.

Table 8. Data and run flags

Flag	Default	Effect
`--domain DIRS`	auto-detect	One domain directory or a comma-separated list; when omitted, `--getdata` targets all domains in the home, and `--run` requires it (or a run from inside a domain).
`--date YYYYMMDD`	today (UTC)	Initialization date for `--getdata`; validated strictly.
`--cycle HH`	`00`	Synoptic cycle: one of 00, 06, 12, 18.
`--start N`	0	First forecast hour to download.
`--end N`	120 (21 for era5)	Last forecast hour to download.
`--step N`	3	Interval between forecast hours.
`--margin DEG`	1.0	Degrees added around the d01 extent derived from namelist.wps.
`--bbox W E S N`	none	Explicit download area; four separate floating-point arguments.
`--source SRC`	`gfs`	Default source keyword for `--run` when the run arguments do not name one.
`--outdir DIR`	per-domain	Output directory override for `--run` products.

5.7 Automation and Housekeeping

Three flags make the tool scriptable: `--allyes` answers YES to every question (the install interview, dataset offers, and overwrite confirmations), `--dryrun` prints the action plan without touching the network or disk, and `--debug` raises verbosity so every subprocess invocation is visible. Combine them - `--dryrun --debug` - when reproducing a problem

for a bug report. The housekeeping trio `--version`, `--flags`, and `--help` exit immediately and are safe to run anywhere.

5.8 Exit Codes

Scripts and CI pipelines can rely on three outcomes, matching the argparse convention of the original Python tool. Anything not listed here (for example 126 or 127 from a missing interpreter) comes from the shell, not from the tool.

Table 9. Exit codes

Code	Meaning
0	The requested action completed successfully.
1	A fatal error occurred - download failure, missing prerequisite, refused request. The console shows the skull banner and the log directory names the stage.
2	Command-line usage error in argparse style, or the run was interrupted with Ctrl-C.

5.9 Environment Variables

The tool's contract with the environment is short. `WRF_HOME` supplies the default installation root (first command-line flag `--homedir` wins if both are present). `WRF_NP` and `REAL_NP` set the MPI rank counts used by `--run`, defaulting to 8 and 2 respectively; a hyper-threading warning at startup reminds you to size `WRF_NP` to physical cores. `LC_ALL` is forced to `C` internally so compiler messages stay parseable. Finally, `http_proxy`/`https_proxy` are cleared at startup unless re-supplied through `--proxy`, which guarantees identical behavior between interactive and batch shells.

6. Installation Layout

6.1 How the Home Directory Is Chosen

Every path the tool uses hangs off one installation home. It is resolved in a strict order: the `--homedir` flag wins; otherwise `$WRF_HOME`; otherwise, if the current directory (or an ancestor walk via the `wrfall` link) is a domain created by `--mkdomain`, the home recorded in that domain's `.wrfems_domain` metadata; otherwise the interactive install interview asks, defaulting to `~/Build_WRF`. Once resolved, the home is recorded in every generated file, so later invocations from anywhere agree on it.

The home is more than a convenience - it is the unit of coexistence. Releases live next to each other as separate directories, libraries are shared through `<home>/local`, and domains reference the release they were created with. Deleting one release directory never damages another, and `--update` exploits the same structure to build a new version while the old one keeps running.

6.2 Directory Map

Table 10. Top-level layout of the installation home

Path	Created by	Contents
`<home>/LIBRARIES`	install	Unpacked source trees of the nine support libraries, kept for reference and for `--force-libs` rebuilds.
`<home>/local`	install	The compiled stack: `bin`, `lib`, `include` - the equivalent of a `/usr/local` that you own.
`<home>/packages`	install	Downloaded tarfiles, kept by default so reinstalls are network-free (`--scrubpkgs` disables this).
`<home>/etc/modulefiles`	install	Environment-modules file(s) so clusters can `module use` the stack.
`<home>/logs`	install	One log per build stage; the first place to look after any failure.
`<home>/WPS_GEOG`	install	The mandatory terrestrial dataset (unless `--nogeog`).

Path	Created by	Contents
<home>/Scripts	--addpack scripts	The wrapped automation scripts (Table 4) for reference and direct use.
<home>/WRF-<ver>-<Type>	install	One directory per release and build type, e.g. WRF-4.6.0-ARW.
<home>/WPS-<ver>-<Type>	install	Matching WPS trees, e.g. WPS-4.6.0-ARW.
<home>/<domain>	--mkdomain	Run domains (next section).

The type label in directory names comes from the build type: ARW, Chem, Hydro, Fire, or FireIdeal. Installing -arw and -chem for the same release therefore gives four sibling directories, each independently usable, each pointing at the same shared libraries. The modulefile generated under etc/modulefiles exports the netCDF, HDF5, and MPICH environment in one module load.

6.3 Anatomy of a Run Domain

A domain is a self-contained working directory: it holds the WPS executables as symlinks, the namelists, the data sub-directories, and a wrfall link back to the installer. The layout below is created by --mkdomain and is stable across releases, which is what allows --update to refresh domains in place.

Table 11. Contents of a domain directory

Entry	Kind	Purpose
geogrid.exe, ungrib.exe, metgrid.exe	symlink	Into the release's WPS directory, so domain refreshes follow updates.
wrf.exe, real.exe, link_grib.csh	symlink	Into the release's WRF directory; --run verifies their presence.
wrfall	symlink	Back to the installer script; running it inside the domain targets the domain.
namelist.wps, namelist.input	file	The two control files; fire domains carry an extra &fire block (Chapter 9.2).
GFS_Input/, ECMWF_Input/, ERA5_Input/	dir	One YYYYMMDDCC sub-directory per downloaded cycle (Chapter 10.5).
wrf_output/{geogrid,ungrib,metgrid}	dir	Per-cycle archive of intermediate products.

Entry	Kind	Purpose
`wrf_output/<YYYYMMDDCC>/`	dir	Final forecast output, including Plots (Chapter 11).
`.wrfems_domain`	JSON	Machine-readable metadata: home path, creation date, type - read when running from inside the domain.
`domain_env.sh`	file	Sourced shell snippet exporting the stack environment for this domain.

6.4 Release Coexistence and Scouring

Because directories encode release and type, `--update` simply adds new siblings - for example `WRF-4.7.0-ARW` beside `WRF-4.6.0-ARW` - and then rewrites the symlinks inside every domain to the new release unless `--norefresh` was given. Old releases are never touched automatically; add `--scour` when you want the previous directories removed after a verified build. The shared libraries are reused verbatim during updates, which is why an update usually takes fifteen minutes rather than an hour: only WRF and WPS recompile.

6.5 Generated Environment Files

At the end of an install the tool writes the environment templates the Do-List refers to: a shell file exporting `NETCDF`, `NETCDF_CLASSIC`, `LD_LIBRARY_PATH`, and `PATH` additions for `<home>/local/bin`, plus the modulefile described above. Chemistry builds additionally export `FLEX=<home>/local/bin/flex` and the absolute `YACC` path, because KPP's build system must find them (Chapter 8.3). Sourcing the shell file is equivalent to loading the module; do both only if you enjoy redundancy.

7. Building the Library Stack

7.1 The Nine Libraries

WRF's runtime dependencies are famously fussy about versions, which is why the installer pins exact releases rather than trusting system packages. The inventory below is the complete set, in the order they are built - the same order the install banner uses, as the annotated transcript in Section 4.8 shows. All of them install into `<home>/local`, so they coexist peacefully with anything your distribution provides.

Table 12. Pinned library versions and their roles

Library	Version	Role in the stack
zlib	1.2.13	Base compression; HDF5 and netCDF link against it.
HDF5	1.12.2	Scientific data container under netCDF4; built with the HL API and Fortran bindings, parallel mode off.
netCDF-C	4.9.2	The C netCDF API; DAP and byterange disabled for a lean, hermetic build.
netCDF-Fortran	4.6.1	The Fortran API WRF actually calls; linked with `-lnetcdf -lhdf5_hl -lhdf5 -lz`.
MPICH	4.1.2	MPI implementation providing `mpicc`, `mpif90`, and `mpirun` used by WRF and `--run`.
libpng	1.6.39	PNG support required by WPS graphics utilities.
Jasper	1.900.1	JPEG-2000 codec that ungrib needs to decode GRIB2 pressure fields.
flex	2.6.4	Lexer generator required by KPP in WRF-Chem builds.
byacc	current	Parser generator; flex's companion for KPP grammar files.

7.2 Build Order and Why It Matters

The order is a dependency chain, not a preference: HDF5 needs zlib, netCDF-C needs HDF5, netCDF-Fortran needs netCDF-C, and WRF needs all of them plus MPICH. WPS adds libpng and Jasper for GRIB2 decoding. Chemistry builds append flex and byacc because the KPP preprocessor is generated code. Each library is configured with explicit `--with-...=<home>/local` paths, so the chain never accidentally picks a system library - the classic source of “works on my login node, fails on the compute node” bugs.

Verification is built in: after the netCDF pair compiles, the tool exercises the Fortran interface with a small test program before declaring the stack healthy, and Chemistry builds additionally run a flex/yacc smoke test. A failure at any stage stops the process with the skull banner and points at the corresponding log; because stages are idempotent, fixing the problem and re-running with `--continue` skips everything already built.

7.3 The Parallel Download Engine

Large tarballs - the 30 GB WPS_GEOG set above all - are fetched with HTTP range requests split across `--dl-jobs` workers (forked processes in the Perl edition). Each worker writes its own segment file plus a `.pNNN` progress marker; a `.eNNN` file records a failed segment and an `.abort` file stops the crew cleanly when you press Ctrl-C. On the next run, completed segments are kept and only the gaps are refetched - resume granularity is the segment, not the whole file.

Practical guidance: the default of 8 jobs suits most broadband links; increase it for datacenter connections and reduce it when a server throttles. `--getdata` caps the value at 3 regardless of your setting, as a courtesy to the public data servers. `--dl-jobs 1` selects the single-stream path, which is occasionally faster on very unreliable links and also uses the classic whole-file resume.

7.4 Local Repositories and Package Reuse

Tarfiles are never deleted by default: they accumulate in `<home>/packages` so that a reinstall, a second build type, or a colleague on the same filesystem can reuse them without a single byte of re-download. Three mechanisms build on this. `--repodir DIR` points at any directory of tarfiles - a USB disk, a shared project folder - and the installer uses it in place of the network. `--nolocal` ignores `<home>/packages` when you suspect a corrupted file and want a fresh copy. `--scrubpkgs` inverts the default after an install, deleting the tarfiles on disk-constrained machines.

7.5 Rebuild Strategies

Choosing between the recovery flags is simple once you know what each preserves. Use `--continue` after any interruption: it keeps every artifact that exists. Use `--force` when a WRF/WPS build succeeded mechanically but you changed your mind about the configure choice - it recompiles the models while keeping the library stack. Use `--force-libs` only when a library itself was mis-built or you changed toolchains; it wipes and rebuilds the nine libraries from the tarballs already in `<home>/packages`, which is normally a matter of minutes since nothing needs re-downloading.

8. Compiling WRF and WPS

8.1 Configure Choices Made for You

WRF's interactive `./configure` menu is answered without you: option 34 - x86_64 with gfortran and dmpar (distributed-memory MPI) - for WRF, nesting option 1, and option 3 for WPS. These are the community's standard choices for a workstation or a single compute node. Sites that prefer shared-memory builds can override the numbers with `--wrf-opt`, `--nest-opt`, and `--wps-opt` (Chapter 5.5); a `--dryrun` shows the exact answers beforehand. After configure, the tool inspects the generated `configure.wrf` and, for Chemistry builds, re-points the `YACC`/`FLEX` assignments to the freshly built local binaries so KPP's code generation uses known-good tools.

8.2 Build Types in Depth

All types share the same library stack and the same environment machinery; they differ in what is downloaded and which compile target is invoked.

- **ARW** compiles `em_real`: the full real-data case with all executables (`wrf.exe`, `real.exe`, `ndown.exe`, `tc.exe`, and friends). WPS is compiled with the three standard programs.
- **Chem** also compiles `em_real` but builds the KPP preprocessor first, needing flex and byacc; after WRF, it offers the eight chemistry geographic datasets for `--addgeog chem`.
- **Hydro** fetches NCAR's wrf_hydro_nwm_public v5.3.0 alongside the ARW tree and compiles the Hydro coupling layer; the resulting `wrf_hydro.exe` pairs with a standard `wrf.exe`.
- **Fire** compiles `em_real` like ARW but prepares WPS for the fire grid - the GEOGRID.TBL gains NFUEL_CAT and ZSF entries and `namelist.wps` gains the subgrid ratios (Chapter 9.2).
- **FireIdeal** compiles the ideal cases `em_fire` (hill_simple, two_fires at your choice) and skips WPS entirely - no real data is needed.

8.3 Chemistry Builds: flex, byacc, and KPP

KPP - the Kinetic PreProcessor that turns chemical mechanism descriptions into Fortran - is the delicate part of a WRF-Chem build, and version 1.4.2 hardened the whole path. Before compiling, the tool verifies that `yacc`, `flex`, and the `libfl` development file are usable; it builds both tools locally anyway, so a broken distribution flex cannot interfere. During the build it exports `FLEX=<home>/local/bin/flex`, and after `./configure` it rewrites the

`YACC/FLEX` lines in `configure.wrf` to absolute local paths - necessary because the assignments in that file would otherwise override the exported environment.

If KPP still fails, the tool's hint distinguishes three signature cases: the `kpp` binary never being built, an apt-installed flex that crashes, and the classic “Error 127 / third body ... M” failure of older toolchains. It also prints a standing warning for WRF 4.7 and newer: a known upstream build bug (wrf-model/WRF issue #2259) breaks the KPP build there, so Chemistry users should stay on 4.6.x until upstream ships the fix.

8.4 gfortran 10 and Newer

Modern gfortran turns WRF-Chem 4.7's mixed-type argument passing into hard errors. The tool detects the combination - Chemistry, WRF 4.7 or newer, gfortran 10 or newer - and adds `-m64 -fallow-argument-mismatch` to the compiler flags automatically. WRF 4.6.x builds are deliberately left untouched because they compile cleanly without it; if you need the flag elsewhere for site reasons, pass it through your own `FCFLAGS` and let the tool's configure answers do the rest.

8.5 What You End Up With

A completed ARW build leaves `wrf.exe`, `real.exe`, `ndown.exe`, `tc.exe`, and the data-assimilation executables in the WRF directory, and `geogrid.exe`, `ungrib.exe`, `metgrid.exe` in WPS. The domain creation step (next chapter) links the handful you actually run into each domain, so operators never edit release directories again after the build. Compile time on an 8-core machine is roughly 25 minutes for ARW and 40 to 60 for Chem or Hydro; the logs show per-stage timings if you want the numbers for your hardware.

9. Creating Run Domains

9.1 The --mkdomain Workflow

A domain is where real work happens, and `--mkdomain` builds one in a few seconds:

```
wrfall_install.pl --mkdomain ethiowrf -all      # -all resolves to the
first type (ARW)

wrfall_install.pl --mkdomain firedom -fire      # WRF-Fire variant
```

Because several builds coexist after a `-all` install, the type qualifier matters. With `-all` (or any explicit type) the choice is made for you - `-all` resolves to the first type of its expansion, ARW. With several builds installed and **no** type flag at all, the command stops and lists what it found, ending with "Tell me which one this domain is for" and an example command; naming the type explicitly (`-arw`, `-chem`, `-fire`) is the unambiguous habit worth forming. The transcript below is the complete screen output of the first command, captured on the reference machine (user `goitomk`, home `/home/goitomk/Build_WRF`):

```
$ ./wrfall_install.pl --mkdomain ethiowrf -all


    Welcome to WRFall. You are creating the domain 'ethiowrf'.
      ✓  /home/goitomk/Build_WRF/Scripts already filled


    ✓  Domain ready: /home/goitomk/Build_WRF/ethiowrf
      Layout:  GFS_Input/  ECMWF_Input/  ERA5_Input/  (each  holds
YYYYMMDDCC/), wrf_output/{metgrid,ungrib,Plots,geogrid}/,
               wrfall -> /home/goitomk/book/wrfall_install.pl
      Shared  scripts  (utility  scripts  +  WRF-domain-Wizard):
/home/goitomk/Build_WRF/Scripts
      Edit namelist.wps first (e.g. with WRF-domain-Wizard): --getdata
reads its lat/lon extent
      and downloads only that area (+ --margin degrees).
      Work from inside the domain:
        cd /home/goitomk/Build_WRF/ethiowrf
        ./wrfall --getdata gfs --date YYYYMMDD --cycle 00 --end 72
        ./wrfall --run auto gfs
      Load       its       variables       with:              source
/home/goitomk/Build_WRF/ethiowrf/domain_env.sh


    WRFall finished. You have created the domain 'ethiowrf'.
```

The welcome line names the action being performed, and the first check-mark comes from the shared-scripts step: `--mkdomain` ensures the utility scripts and, when installed, the WRF-domain-Wizard are present under `<home>/Scripts` - “already filled” on any run after the first. The **Domain ready** line announces completion, and the **Layout** lines that follow are a compressed map of what Chapter 6.3 documents in full: one input tree per source with a `YYYYMMDDCC` sub-directory per cycle, the four-fold `wrf_output` archive, and the `wrfall` soft link, which on this machine points back to `/home/user/wrfall_install.pl` - the script location chosen in Chapter 3.1, recorded as an absolute path so the domain works from anywhere.

The guidance lines are the tool's own quick-start, worth reading once slowly. “Edit namelist.wps first” is the operational contract of Chapter 10.2: `--getdata` derives the download area from the d01 extent in that file, plus `--margin` degrees, so the domain geometry you set is exactly the area it fetches. The **Work from inside the domain** block is the daily loop - change into the domain and use its `wrfall` link, which targets that domain and its recorded home without further flags. “Load its variables with: source ...domain_env.sh” is for shells where you want the stack environment (`WRF_HOME`,

WRF_WPS, WRF_RUN, DOMAIN_DIR) available to your own commands. The closing line - "WRFall finished. You have created the domain 'ethiowrf'." - mirrors the welcome and confirms the actions completed. Note what did **not** happen: creating the domain does not run geogrid - that happens either manually or as the optional first stage of `--run auto`.

9.2 Fire Domains

The `-fire` qualifier turns the domain into a WRF-Fire workspace using the level-set fire module (ifire = 2) that ships inside every WRF-ARW build. Three additions distinguish it from a plain domain. First, `namelist.wps` gains the subgrid ratios `sr_x = sr_y = 10` (they must be even) plus the matching `subgrid_ratio_x/y` entries. Second, the GEOGRID.TBL entries are extended with the two fire fields: **NFUEL_CAT**, a categorical fuel-type field interpolated from the `landfire/` folder, and **ZSF**, a high-resolution terrain field from `highres_elev/`, both marked `subgrid=yes`. Third, `namelist.input` receives a complete `&fire` block with the level-set switch and a one-ignition template (a 50-meter circle at grid position 1000, 1000 ignited 600 seconds in) that you edit per scenario.

The fire geographic data is not downloaded automatically: `--addgeog fire` creates the two folders `landfire/` and `highres_elev/` inside the domain's WPS_GEOG area and expects you to place the Landfire and elevation assets there, since they are typically obtained from agency portals with site-specific licensing. `halt_on_missing` is set in the geogrid entries, so geogrid fails loudly rather than silently producing empty fire fields.

9.3 Editing the Domain

Two files control everything. In `namelist.wps` you set the map projection, the domain extent, grid dimensions, and nesting ratios - the Domain Wizard package (`--addpack domwiz`) provides a graphical editor if you prefer clicking to counting grid points. In `namelist.input` you set physics, time control, and output cadence; `--run` rewrites only the dates it owns (Chapter 11.3) and keeps a timestamped backup before doing so. The metadata file `.wrfems_domain` should not be hand-edited; it is the contract between the domain and the installer.

9.4 Importing and Refreshing Domains

Domains survive releases. `--import DIR` (repeatable) pulls directories containing a `namelist.wps` from an older installation into the new home, re-linking their executables to the current release. After an `--update`, every domain in the home is refreshed the same way unless `--norefresh` was passed; refresh rewrites the symlinks and re-applies the fire blocks where applicable, leaving your namelist edits and data untouched.

9.5 The Wizard and the Utility Scripts

Two optional packages round out the domain experience. `--addpack domwiz` clones the WRF-domain-Wizard repository - a graphical domain editor - and `--addpack scripts` installs the original automation scripts under `<home>/Scripts` for reference or direct use, including the underlying `setup_wrf_domain.sh`, `run_wrf.sh`, and the three data downloaders that the modern flags replace. Long-time users appreciate having both paths available; the flag-based path remains the supported one.

Tip. `--domain` accepts a comma-separated list, and `--getdata` with no `--domain` at all targets every domain in the home. Overnight GFS fetches for four domains are one command: `wrfall_install.pl --getdata gfs --date 20261016 --end 72`.

9.6 Naming and Organizing Multiple Domains

Domain names become directory names, so choose them for six-months-from-now you: `ethiowrf` says more than `dom1`, and a region-plus-purpose pattern - `highlands72`, `fire_test_bed` - scales well. Because every command accepts several domains, grouping by workflow works naturally: a forecast suite can be `plain`, `smoke`, and `verify`, fetched together with one `--getdata` and run sequentially with one `--run` when a comma-separated `--domain` list is given. Keep the names free of spaces; the comma-list syntax makes the comma the only reserved character.

Under the hood each domain is independent, so you can safely delete one by removing its directory, archive one by tarring it (the `wrfall` link and `domain_env.sh` regenerate on import), and duplicate one by copying plus re-running `--mkdomain` with the new name to rebuild the links. When you outgrow one home - say a production home and an experimental one - install into two `--homedir` roots and treat each as a complete world; the recorded `.wrfems_domain` metadata keeps both unambiguous.

10. Downloading Meteorological Data

10.1 The Three Sources

`--getdata` supports the three sources most operational and teaching workflows need, all at 0.25-degree resolution. GFS is the global forecast model of NOAA - the default and the fastest to fetch. ECMWF IFS open data covers the same use cases from the European model. ERA5 is the reanalysis: slower to obtain through the Copernicus queue, but the authoritative choice for case studies and verification work. The source also decides the default forecast depth - 120 hours for GFS and ECMWF, 21 hours for ERA5 - overridable with `--end`.

Table 13. Data sources and their characteristics

Source	Provider	Access path	Notes
gfs	NOAA NOMADS	Direct HTTPS GRIB2	Default source; cycles 00/06/12/18 available a few hours after synoptic time.
ecmwf	ECMWF open data	Direct HTTPS GRIB2	IFS 0.25° open-data product; same layout as GFS under the domain.
era5	Copernicus CDS	Python cdsapi queue	Requires `--addpack cdsapi` (pip --user) and a free CDS API key; requests are queued server-side.

10.2 Choosing the Area

The download area follows the domain by default: the tool reads the d01 latitude/longitude extent from the domain's `namelist.wps`, adds `--margin` degrees on every side (default 1.0), and snaps the rectangle outward to the 0.25-degree grid. This guarantees ungrib never runs off the edge of the data while downloading as little as possible. Two overrides exist: `--bbox W E S N` supplies an explicit rectangle in the order west, east, south, north - four separate floating-point arguments - and running `--getdata` with no domain at all falls back to a documented default area (latitudes -10 to 25, longitudes 20 to 54) with a warning, so a forgotten `--domain` never silently downloads the wrong region.

10.3 Controlling Time

Time selection uses five flags: `--date YYYYMMDD` (default: today in UTC), `--cycle` one of 00/06/12/18 (default 00), `--start` first forecast hour (default 0), `--end` last forecast hour (source-dependent, see above), and `--step` the interval (default 3 hours). The combination is validated strictly - argparse-style errors for malformed dates and cycles. ERA5 adds a subtlety handled transparently: its surface analysis fields cross midnight, so

the client performs the day rollover for you when a range ends past 00 UTC of the following day.

```
# GFS 00 UTC cycle, 0..72 h at 3-h steps, for one domain
wrfall_install.pl --getdata gfs --domain ~/Build_WRF/ethiowrf \
                  --date 20261016 --cycle 00 --end 72

# Explicit area instead of the domain's namelist.wps
wrfall_install.pl --getdata ecmwf --bbox 32.0 48.0 3.0 15.0 \
                  --date 20261016 --end 48
```

10.4 ERA5 Prerequisites

ERA5 downloads use the official Copernicus client, which is Python. `--addpack cdsapi` installs it into your user site-packages with pip and reminds you to place your CDS API key in `~/.cdsapirc`. Requests then queue on the Copernicus servers; the tool submits and waits, printing progress. Because the queue can be long, ERA5 is a background job by nature - submit it, collect the files later, and let `--run` verify completeness before the forecast. The same politeness cap applies: at most three parallel connections.

10.5 Storage Layout and Reuse

Files land under the domain in source-specific trees: `GFS_Input/`, `ECMWF_Input/`, or `ERA5_Input/`, each containing a `YYYYMMDDCC` sub-directory per cycle. Re-running the same command skips already-downloaded hours, and the downloader streams GRIB records to disk rather than buffering files in memory, so even 120-hour fetches have a small footprint. `--run` consumes these directories in place; nothing is copied or renamed between the download and run stages, which keeps the two steps independently schedulable.

10.6 Verifying What You Downloaded

Before spending an hour of model time on incomplete data, spend ten seconds on verification. The downloader leaves a per-cycle directory per source - for example `GFS_Input/2026101600/` - and completeness has a simple shape there: the expected number of GRIB files for your `--start`/`--end`/`--step` window, each larger than zero bytes, and no `.eNNN` segment-error files beside them. Re-running the same `--getdata` command is always safe: finished hours are skipped, gaps are refetched, and the summary lines state exactly what was reused and what was fetched.

Two failure signatures are worth recognizing on sight. A directory that exists but holds fewer files than expected means an interrupted run - simply repeat the command. A directory full

of small files of a few hundred bytes means the upstream server answered with error pages instead of GRIB data, which happens when a cycle is not yet published; wait for the source's publication lag and fetch again. `--run` re-checks the directory anyway and names the exact `--getdata` command if anything essential is missing.

11. Running WRF End to End

11.1 The --run Command

`--run` is the daily driver: one command executes link_grib, ungrib, metgrid, real, and wrf in order, with the right environment and the right namelist dates. Its arguments come in two forms - the fully automatic one for routine operations, and the explicit one for reruns and case studies:

```
wrfall_install.pl --run auto --domain ~/Build_WRF/ethiowrf
wrfall_install.pl --run 20261016 00 72 gfs --domain ~/Build_WRF/ethiowrf
cd ~/Build_WRF/ethiowrf && ./wrfall --run 20261016 12 48
```

The first form takes today's date (UTC), cycle 00, and a 72-hour forecast. The second names the date, cycle, and forecast hours explicitly, with an optional trailing source keyword that overrides `--source`. Forecast hours must be between 1 and 72. As with `--getdata`, the `--domain` flag accepts a comma-separated list, and running from inside a domain directory via its `wrfall` link implies the domain.

11.2 The Pipeline, Stage by Stage

Before anything executes, `--run` verifies the five required files in the domain - `ungrib.exe`, `metgrid.exe`, `real.exe`, `wrf.exe`, `link_grib.csh` - and that the GRIB input directory for the requested date and cycle exists, telling you the exact `--getdata` command if it does not. The stages then run in sequence, each with its products archived under `wrf_output/`:

- **link_grib + ungrib** - Vtables are selected per source, GRIB records are decanted into intermediate files, archived under `wrf_output/ungrib/<YYYYMMDDCC>`.
- **metgrid** - horizontal interpolation onto your nest(s), archived under `wrf_output/metgrid/<YYYYMMDDCC>`.
- **real** - vertical initialization producing `wrfinput_d01` and `wrfbdy_d01`, running on `REAL_NP` MPI ranks (default 2).
- **wrf** - the forecast itself on `WRF_NP` ranks (default 8) via `mpirun`; products move to `wrf_output/<YYYYMMDDCC>/` on completion.

Namelist dates are rewritten from the command arguments: `--run 20261016 00 72` sets `start_date`/`end_date` and the run hours across domains, keeping a timestamped backup of your `namelist.input` first. Everything else in the namelist is yours - physics, output cadence, restart intervals - and is left untouched.

11.3 Monitoring a Run

WRF reports through the `rsl.*` files: `rsl.error.0000` and `rsl.out.0000` carry rank zero's diary, including the timing lines for each domain and the final SUCCESS marker. A 60-hour forecast on a typical 8-rank workstation takes one to three hours depending on grid size; watch progress with `tail -f rsl.error.0000` or check `timing` lines in `rsl.out.0000`. If the run dies, the last lines of rank zero plus the matching `wrf.log` almost always name the culprit namelist entry - the troubleshooting table in Chapter 13 collects the common patterns.

11.4 Rank Counts and Hyper-threading

The two rank counts are environment variables, not flags, so cluster jobs can set them per allocation: `export WRF_NP=16 REAL_NP=4` before `--run`. The installer warned you at startup if hyper-threading was on, and the same advice applies here - one MPI rank per physical core is the efficient configuration, oversubscribing usually slows WRF down. For fire runs, keep at least one rank free of the ignition region tuning: the fire module is latency-sensitive and benefits more from fewer, faster ranks than from a larger count.

11.5 Running Fire Cases

A `-fire` domain runs exactly like an ARW domain - same pipeline, same flags - once its `&fire` namelist block is tuned. The template created by `--mkdomain firedom -fire` defines one circular ignition: `fire_ignition_start_x1/y1` and the matching end coordinates at 1000 meters, `fire_ignition_radius1` of 50 meters, ignited at `fire_ignition_time1` = 600 seconds. Change those four numbers per scenario, leave `ifire = 2` and `fire_fuel_read = -1` (fuel categories come from the NFUEL_CAT field geogrid produced), and the run proceeds through the standard `--run` command. `fire_print_msg = 1` writes the fire diagnostics you will want when validating a new ignition.

11.6 Interrupted Runs and Reruns

`--run` is deliberately restart-from-scratch: it rewrites the dates, re-runs the pipeline, and produces a fresh, self-consistent output set, which is the safest semantic for an operations tool. An interrupted forecast is therefore retried by re-issuing the same `--run` command - ungrib and metgrid products are regenerated in minutes, and real plus wrf start clean. Genuine WRF warm restarts (continuing a long run from `restart_d01` files with `restart = .true.`) remain a namelist-level operation: set the restart fields in `namelist.input` yourself, run `wrf.exe` directly, and leave the `--run` wrapper for the cold-start cases it is designed for.

Reruns with changes follow the same pattern. To extend a forecast from 48 to 72 hours, make sure the extra GRIB hours were downloaded (`--getdata` with the larger `--end`), then re-run with the longer horizon; the wrapper refreshes `end_date` and the run hours from the command line. Each rewrite keeps a timestamped backup of your previous `namelist.input`, so comparing what changed after a week of experiments is a `diff` away.

12. Updates and Maintenance

12.1 Updating to a Newer Release

`--update` is the calm way to move forward. The tool locates the current release from the existing directories, asks the GitHub API for newer releases (falling back to its built-in list offline), builds the chosen one beside the old, reuses every compiled library, and then refreshes all domains in the home to point at the new release. A specific target can be named - `--update 4.7` - and `--norefresh` keeps domains frozen on the old release if you prefer to migrate them deliberately. Because libraries are reused, an update typically recompiles only WRF and WPS.

Rolling back is the mirror image: `--install` an older release requires `--force`, and afterwards the domains can be re-pointed with `--import` or a plain refresh. Nothing in the design assumes the newest release is the right one - the tool simply records what exists and lets you choose.

12.2 Multiple Build Types Side by Side

Adding a capability later is one command: `--install -chem` on a machine that already has ARW builds the Chemistry tree (and its flex/byacc extras) while reusing the entire library stack, then offers the chemistry datasets. The same applies to `-hydro` and `-fire`. Each type lands in its own labeled directories, so disk usage grows per type but nothing is duplicated between them.

12.3 Housekeeping

Three regular chores keep an installation tidy. First, prune old releases once a new one is verified - `--scour` after an update, or a manual delete of the old `WRF-*/WPS-*` directories, which never affects shared libraries. Second, decide your tarfile policy: keeping `<home>/packages` costs roughly two gigabytes but makes every future operation network-free; `--scrubpkgs` in automated pipelines trades that for disk. Third, glance at `<home>/logs` occasionally - it grows one file per stage and is safe to truncate at any time.

12.4 Moving or Removing an Installation

The whole installation is a plain directory tree owned by you, so it copies, archives, or deletes as one unit. To move it, copy `<home>` and update the paths written inside the env file, the modulefile, and each domain's `domain_env.sh` - or simply re-run `--update --norefresh` in the new location to regenerate them. To remove everything, delete `<home>` and drop the two lines the tool added to `~/.bashrc` (the `WRF_HOME` export and the env

sourcing line). Because nothing was ever installed outside the home, that is the entire uninstall.

13. Troubleshooting and FAQ

13.1 A General Debugging Strategy

Almost every problem yields to the same four steps. Reproduce with `--dryrun` to confirm what the tool intends to do. Read the stage log under `<home>/logs` - console messages are summaries, logs are evidence. Check the prerequisites table in Chapter 2 before suspecting the tool itself: disk full, missing gfortran, and unreachable hosts cover the majority of first-week failures. Then re-run with `--continue`; resumability means a diagnosed failure rarely costs more than the failed stage.

13.2 Common Failures and Fixes

Table 14. Frequently observed errors, their causes, and remedies

Symptom	Likely cause	Fix
Download stalls at 99% or segments fail repeatedly	Unstable connection; throttling CDN	Re-run with `--continue`; try `--dl-jobs 4` or `--dl-jobs 1`; pin `--wget` or `--curl`.
“argument --curl: ignored explicit argument” style errors	Flag typo or attached value	Read the argparse message literally; check spelling against `--flags`.
`--update` refuses: nothing installed	No previous release in the home	Run `--install` first; updates build next to an existing release.
“Pick one of --curl or --wget”	Both transport flags given	Keep one, or omit both for auto.
netCDF Fortran test fails during library stage	Mismatched compiler or stale LD_LIBRARY_PATH	Unload conflicting modules; re-run with `--continue`; inspect `logs/netcdf_fortran*`.
KPP build fails; `kpp` binary missing or Error 127	Broken flex/yacc, or the WRF 4.7 upstream bug	Stay on WRF 4.6.x for Chem (issue #2259); the tool's local flex/byacc are used automatically - see its hint text.

Symptom	Likely cause	Fix
gfortran 10+ “type mismatch in argument” errors	Chemistry ≥ 4.7 without argument-mismatch flag	The tool adds `-fallow-argument-mismatch` automatically for that exact combination; verify gfortran version detection in the log.
“More than one build is installed under … Tell me which one this domain is for”	`--mkdomain` without a type flag after a multi-type (`-all`) install	Name the build the domain targets: `--mkdomain mycase -arw` (or `-chem`, `-hydro`, `-fire`). `-all` itself resolves to ARW.
“Required file not found in the domain: wrf.exe”	Domain created before the release finished, or stale symlinks	Re-run `--mkdomain` import/refresh, or recreate the domain; check `--update` completed.
“GRIB input directory not found: …”	`--run` before `--getdata`, or wrong date/cycle	Run the printed `--getdata` command; confirm the `YYYYMMDDCC` folder exists.
ungrib complains about missing fields	Area smaller than d01 or wrong Vtable for source	Re-download with a larger `--margin`, or explicit `--bbox`; confirm the source keyword.
`mpirun` fails with too many ranks	`WRF_NP` exceeds physical cores	Set `export WRF_NP=<physical cores>`; avoid hyper-threaded oversubscription.
Disk fills during install	Planning figures exceeded	Check Table 4 budgets; add `--nogeog` with a shared WPS_GEOG, or `--scrubpkgs`; monitor with `df -h`.

13.3 Frequently Asked Questions

Do I need root for anything? No. The one administrative ask is having gfortran, wget or curl, and roughly 40 GB of quota. Everything else - libraries, WRF, WPS, data, modulefiles - lives under `<home>`, and the tool never writes outside it.

Can I install on a cluster with environment modules? Yes. The install generates a modulefile under `<home>/etc/modulefiles`; add the directory with `module use` and load it on compute nodes. Set `WRF_NP`/`REAL_NP` per allocation before `--run`.

Why is my download slow even though the link is fast? Public data servers and some CDNs throttle per connection. Raising `--dl-jobs` helps for datasets (cap 3 for `--getdata`), and `--repodir` from a nearby mirror helps most of all.

Can two installs coexist on one machine? Yes - they are just directories. Give each its own `--homedir`, or keep one home and add build types; both models are supported by design.

Does `--install` delete my old release? Never automatically. Releases coexist; you remove old ones with `--scour` or manually when you are ready.

What exactly does `-all` install, and what does it do for `--mkdomain`? For `--install`, `-all` expands to the four real-data build types - ARW, Chem, Hydro, Fire (FireIdeal stays opt-in) - and the console quotes the combined disk figure, about 60 GB with the geographic datasets. For `--mkdomain NAME -all`, it selects the build the domain targets, resolving to the first type of the expansion, ARW; Section 9.1 shows a complete captured example.

How do I run two domains at once? They are independent directories; launch `--run` in each, or pass a comma-separated `--domain` list to process them sequentially in one command.

Where did my wrfout files go? Under `<domain>/wrf_output/<YYYYMMDDCC>/`; intermediate ungrib and metgrid products are archived under the matching sub-folders so reruns start clean.

The tool talks strangely. Is that normal? Yes - the greetings, the skull on fatal errors, and "Ouch!" on Ctrl-C are part of the original tool's personality. Everything after the joke is actionable: the line above the skull names the real problem.

Which WRF version should I pick? The default (4.6.0) is the tested baseline - required for Chemistry until upstream fixes the 4.7 KPP build bug. Other releases install with `--install <version>` when your research needs them.

Python is missing - can I still use everything? Except ERA5, yes. GFS and ECMWF downloads, all builds, domains, and runs are pure Perl and shell; the cdsapi client is only needed for `--getdata era5`.

13.4 Getting Further Help

When a problem resists the playbook above, gather three artifacts before asking: the output of `--version`, the failing stage's log from `<home>/logs`, and the exact command line. The WRF users' forum (forum.mmm.ucar.edu) covers model-side questions, the WRF GitHub repository tracks release-specific issues such as the KPP bug of 4.7, and your team's `Scripts` package documents the workflows this tool automated. Bug reports that include the three artifacts typically get answered in the first exchange.

Appendix A. Complete Flag Reference

This appendix reproduces the full option table exactly as `--flags` prints it, with the chapter that discusses each flag in depth. Defaults: release 4.6.0; `--dl-jobs` 8; `--wrf-opt` 34; `--nest-opt` 1; `--wps-opt` 3; `--cycle` 00; `--step` 3; `--margin` 1.0; `--source` gfs.

Table 15. Every option understood by wrfall_install.pl 1.4.2

Flag	Meaning	See
`--install [REL]`	Fresh install of WRF/WPS plus libraries; REL may be a version or list | listall | listgeog.	5.1
`-arw -chem -hydro -fire -fireideal -all`	Build types; repeatable; `-all` = arw, chem, hydro, fire.	5.2
`--update [REL]`	Build a newer release beside the current one, reuse libraries, refresh domains.	12.1
`--addpack LIST`	Optional packages: domwiz, scripts, cdsapi.	9.5
`--addgeog LIST`	Terrestrial datasets: chem, mandatory, fire, or any UCAR name.	9.2
`--nogeog`	Skip the base WPS_GEOG download.	5.3
`--import DIR`	Import domains from an older installation; repeatable.	9.4
`--scour`	Delete previous release directories after a successful install/update.	12.3
`--continue`	Resume: skip libraries already built.	7.5
`--force`	Rebuild WRF/WPS even if present; allow older releases; keep existing libraries.	7.5
`--force-libs`	Also rebuild the support libraries.	7.5
`--homedir DIR`	Top-level install directory (default $WRF_HOME or ~/Build_WRF).	6.1
`--repodir DIR`	Take package tarfiles from a local repository.	7.4

Flag	Meaning	See
`--nolocal`	Ignore tarfiles in <home>/packages; download fresh.	7.4
`--nounpack`	Download only; do not unpack or build.	5.3
`--scrubpkgs`	Delete package tarfiles after installation.	12.3
`--norefresh`	With --update, do not refresh existing domains.	12.1
`--curl` / `--wget`	Force one download client (default auto: wget, curl, built-in).	5.4
`--dl-jobs N`	Parallel HTTP range connections per file (default 8; --getdata caps at 3).	7.3
`--proxy HOST:PORT`	Set http_proxy/https_proxy (cleared by default).	5.4
`--wrf-opt N` / `--wps-opt N`	Override configure menu choices (defaults 34 and 3, gfortran dmpar).	8.1
`--nest-opt N`	Override the WRF nesting option (default 1).	8.1
`--allyes`	Answer YES to every question (automation).	5.7
`--dryrun`	Show what would be done; no downloads, no builds.	5.7
`--mkdomain NAME`	Create a run domain under <home>/NAME; add -fire for fire.	9.1
`--getdata gfs|ecmwf|era5`	Download initial/boundary data (see date/cycle/start/end/step/margin/bbox/domain).	10.1
`--run auto|YYYYMMDD CC HH [src]`	Run geogrid(opt)/ungrib/metgrid/real/wrf for --domain.	11.1
`--debug --version --flags --help`	Housekeeping.	5.7

Appendix B. Directory and File Manifest

The complete set of paths a full lifecycle creates, with the command that owns each. `<home>` denotes the installation root, `<dom>` a run domain, and `YYYYMMDDCC` a cycle stamp such as 2026101600.

Table 16. Everything the tool ever writes

Path	Owner	Contents
<home>/LIBRARIES/<lib>-<ver>/	--install	Unpacked library sources (nine libraries, Table 12).
<home>/local/{bin,lib,include}	--install	Compiled stack; PATH and LD_LIBRARY_PATH targets.
<home>/packages/*.tar.gz	--install	Downloaded tarfiles, kept for reuse.
<home>/etc/modulefiles/	--install	Generated environment-modules file.
<home>/logs/	all commands	Per-stage build and download logs.
<home>/WPS_GEOG/	--install / --addgeog	Terrestrial datasets; landfire/ and highres_elev/ for fire domains.
<home>/Scripts/	--addpack scripts	Wrapped automation scripts (Table 4).
<home>/WRF-<ver>-<Type>/	--install	WRF source and executables.
<home>/WPS-<ver>-<Type>/	--install	WPS source and executables.
<home>/<dom>/	--mkdomain	Run domain (Table 8).
<dom>/namelist.wps	user	Domain geometry and WPS control.
<dom>/namelist.input	user + --run	Forecast control; date fields rewritten per run.
<dom>/GFS_Input|ECMWF_Input|ERA5_Input/YYYYMMDDCC/	--getdata	GRIB initial/boundary files per cycle.

Path	Owner	Contents
<dom>/wrf_output/{geogrid,ungrib,metgrid}/YYYYMMDDCC/	--run	Archived intermediate products.
<dom>/wrf_output/YYYYMMDDCC/	--run	Final forecast output (wrfout files).
<dom>/.wrfems_domain	--mkdomain	JSON metadata: home path, type, creation info.
<dom>/domain_env.sh	--mkdomain	Shell environment for the domain.
<dom>/wrfall	--mkdomain	Soft link back to the installer.

Appendix C. Glossary

Table 17. Terms used in this manual

Term	Definition
ARW	Advanced Research WRF - the research/community dynamical core of the WRF model; the default build type.
cdsapi	The Copernicus Data Store Python client used to request ERA5 products; installed by --addpack cdsapi.
dmpar	Distributed-memory parallel configuration (MPI); configure option 34 for WRF with gfortran.
Domain	A run directory created by --mkdomain holding namelists, executable links, data, and output for one modeling setup.
GEOGRID.TBL	WPS table describing which terrestrial fields geogrid interpolates; fire domains add NFUEL_CAT and ZSF entries.
geogrid	WPS program that interpolates static terrestrial data onto the model grid.
GRIB	WMO's Gridded Binary format; the delivery format of GFS, ECMWF open data, and ERA5 products.
HDF5	Hierarchical Data Format library under netCDF4; built by the installer with Fortran bindings.
KPP	Kinetic PreProcessor generating Fortran chemistry solvers; required by WRF-Chem builds.
Level set	The interface-tracking method behind WRF-Fire's fireline representation (ifire = 2).
metgrid	WPS program that horizontally interpolates meteorological fields onto the model grid.
MPICH	The MPI implementation the installer compiles; provides mpicc, mpif90, and mpirun.
namelist	Fortran key-value control blocks; namelist.wps drives WPS, namelist.input drives real and wrf.
netCDF	Self-describing scientific data format; WRF reads and writes it through the Fortran API.

Term	Definition
ungrib	WPS program that decodes GRIB files into intermediate format using Vtables.
Vtable	Variable table telling ungrib which GRIB fields to extract for a given data source.
WPS	WRF Preprocessing System - geogrid, ungrib, and metgrid.
WRF-Fire	The embedded wildfire module of WRF-ARW, enabled via &fire and subgrid fire fields.

Appendix D. Document Information

This guide documents **wrfall_install.pl** version 1.4.2, the Perl edition of the WRF/WPS all-in-one installer, and was generated in October 2026. It supersedes any earlier draft covering versions before 1.4.2, whose Chemistry-build behavior changed materially (flex/byacc handling and the KPP warnings). The tool itself embeds the same information through `--help`, `--flags`, and `--version`.

Table 18. Revision history of this manual

Revision	Date	Scope
1.0	October 2026	First full edition covering wrfall_install.pl 1.4.2: all commands, flags, layouts, workflows, troubleshooting, and appendices.
1.1	October 2026	Added the GitHub obtaining instructions (Chapter 3.1) and three annotated console transcripts captured on the reference machine: `--install listall` (4.7), `--install 4.7.0 -all` (4.8), and `--mkdomain ethiowrf -all` (9.1). Extended the troubleshooting table and FAQ for multi-type (`-all`) installs.

Feedback that corrects a fact, a default, or a workflow step is welcome and improves the next revision. Please quote the chapter and section number, the command you ran, and the version output; changes are batched so that printed copies stay consistent with the tool's behavior in the field.

Appendix E. Quick Reference Card

The ten commands that cover ninety percent of daily use, in the order a typical case needs them. Print this page and keep it beside the terminal; every row is explained in the chapter shown in the last column.

Table 19. The daily-driver commands at a glance

Task	Command	See
Install the standard stack	`wrfall_install.pl --install -arw --allyes`	4.1
Install every build type	`--install 4.7.0 -all` (about 60 GB)	4.8
Preview any action safely	append `--dryrun` to the command	3.2
List installable releases / datasets	`--install list` / `--install listall`	3.4
Inspect the full catalog	`--install listall` (annotated in 4.7)	4.7
Add another model variant	`--install -chem` (or -hydro, -fire)	4.6
Create a run domain	`--mkdomain mycase` (add `-fire` for fire)	9.1
Fetch forecast data	`--getdata gfs --date 20261016 --end 72`	10.3
Fetch with an explicit area	`--getdata ecmwf --bbox W E S N`	10.2
Run the whole forecast	`--run 20261016 00 72` (or `--run auto`)	11.1
Watch a running forecast	`tail -f rsl.error.0000`	11.3
Move to a newer release	`--update` (add `--norefresh` to keep domains)	12.1

Two habits make everything smoother. First, run from inside a domain via its `wrfall` link whenever possible - every command then targets that domain without flags. Second, when anything behaves unexpectedly, prefix the command with `--dryrun --debug` and read the plan it prints; the overwhelming majority of surprises are visible there before a single byte moves.

Appendix F. WRF / WPS Domain Generator & Wizard — User Guide

Editorial note (reorganization): the WRF / WPS Domain Generator & Wizard manual, which was embedded in the source file between Chapters 9 and 10, is reproduced verbatim in this appendix so the numbered chapters run continuously.

Web port for **WRF-ARW · WRF-Chem · WRF-Fire · WRF-Hydro / HRLDAS** namelist generation

> **NOTE** Every screenshot in this guide was captured from the application itself using the built-in **Load Example** data (a 3-domain nest over north-east Africa). Coloured boxes and numbers were added to the screenshots; they are not part of the program.

Document date: 25 October 2026

Contents at a glance

Follow the steps in order the first time. Each step names the tab you need and ends with a short check so you know it worked.

Step	Tab	What you do
Step 1	—	Open the application and learn the layout
Step 2	Domain Configuration	Start a project: New, Open… or Load Example
Step 3	Domain Configuration	Choose projection, number of domains and simulation timing
Step 4	Domain Configuration	Enter domain extents and resolutions
Step 5	Domain Configuration	Enter output and geography data paths
Step 6	Domain Configuration / Model Components	Switch on Chem, Fire and/or Hydro components
Step 7	Model Components & Namelists	Edit the &chem, &fire, hydro and HRLDAS parameters
Step 8	Interactive Domain Wizard	Check and adjust domains on the map
Step 9	Namelists & Output	Generate, validate and read the namelists

Step	Tab	What you do
Step 10	Namelists & Output	Download files, save the project and run WRF
Appendix A	—	Worked example (values from the built-in example)
Appendix B	—	Parameter reference for &chem, &fire, hydro and HRLDAS
Appendix C	—	Troubleshooting

What the program does

The application is a single web page that turns a graphical domain definition into the text files needed by the WRF modelling system:

- `namelist.wps` – WPS pre-processing (geogrid, ungrib, metgrid).
- `namelist.input` – WRF run-time settings, with `&chem` and `&fire` blocks added when those components are switched on.
- `hydro.namelist` – WRF-Hydro routing (`&HYDRO_nlist`).
- `namelist.hrldas` – offline HRLDAS / Noah-MP driver.

It runs entirely in your browser. **Nothing is written to disk until you click Download** or Save Domain…

> **IMPORTANT** Generated files are **templates**. The program itself reminds you to verify them against the WRF release you have installed – option names and numbers differ between versions.

> **NOTE** Reading `geo_em.d0X.nc` files and plotting terrain are desktop-only features. In the browser the buttons **Plot Terrain** and **Load geo_em** are greyed out. Use a saved `.json` project or an existing `namelist.wps` with **Open…** instead.

How to read the screenshots

Each screenshot carries **numbered badges**. The same number appears in the table below the figure, which explains the highlighted item. The **colour** tells you what kind of action or information it is:

1	**BLUE** Click	A button, tab, tick-box or menu you click.

2	**GREEN** Type / choose	A field or drop-down where you enter or select a value.
3	**ORANGE** Important	A setting or hint that is easy to get wrong, or a helpful tooltip.
4	**RED** Caution	Deletes or resets something, or marks a warning.
5	**PURPLE** Result	Something the program fills in or displays for you – read it, do not type in it.

Boxed notes use the same idea:

> **TIP Tip** – a shortcut or good practice.

> **IMPORTANT Important** – read before continuing.

> **CAUTION Caution** – can lose work or produce a wrong namelist.

> **NOTE Note** – background information.

Before you start

- A current desktop browser (Chrome, Edge or Firefox). Open the HTML file by double-clicking it.
- The simulation period and the interval of your meteorological input (for example 6 hours).
- The latitude/longitude limits of each domain – or a rough idea you will refine on the map.
- The location of your `WPS_GEOG` static data and of your run directories (you can type these later).
- For Fire, Hydro or HRLDAS runs: the names of the input files you will use (the program leaves grey `/path/to/…` placeholders until you replace them).
- Internet access is needed **only** for the optional map basemap tiles.

STEP 1 Open the application and learn the layout

Open the HTML file in your browser. The window has a dark-blue header, a row of four tabs, the working area, and a status bar at the bottom.

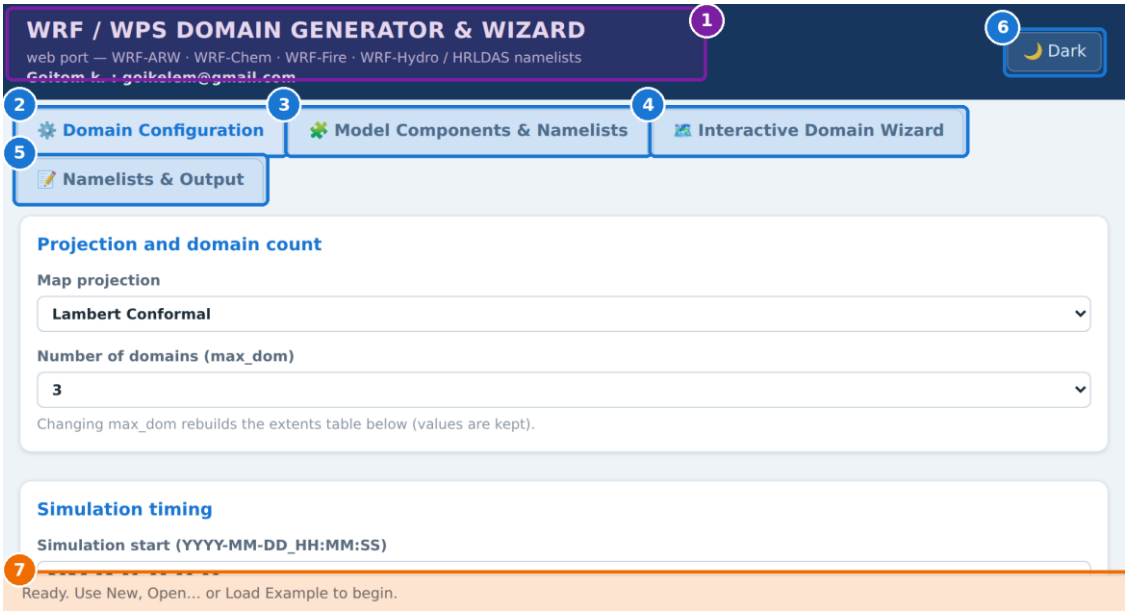


Figure 1. The four tabs, theme switch and status bar

1	Application header: title and project information
2	Tab 1 – Domain Configuration (start here)
3	Tab 2 – Model Components & Namelists
4	Tab 3 – Interactive Domain Wizard (map)
5	Tab 4 – Namelists & Output (generated files)
6	Light / Dark theme switch
7	Status bar – read it after every action; it confirms success or explains problems

> **TIP** Watch the **status bar** (7). After most actions it tells you what happened – for example "Loaded 3-domain example." or "SUCCESS — rendered namelist.wps …".

The tabs follow the work flow: **1** Domain Configuration → **2** Model Components & Namelists → **3** Interactive Domain Wizard → **4** Namelists & Output.

STEP 2 Start a project

Go to the **Domain Configuration** tab and scroll to the **Actions** card.

1. To work from scratch click **New** (2). To continue earlier work click **Open…** (3) and choose a saved `.json` project or an existing `namelist.wps`.
2. To explore with ready-made values click **Load Example** (9). It fills a 3-domain demonstration at 15 km, 5 km and 1.667 km.

3. Check the status bar: it should read “Loaded 3-domain example.”



Figure 2. The Actions card

1	Generate namelists – builds all files and jumps to the output tab
2	New – clears the project (asks first)
3	Open… – loads a saved .json project or an existing namelist.wps
4	Save Domain… – downloads a .json project file
5	Undo the last change
6	Redo
7	Add Domain – appends a nest (raises max_dom by 1)
8	Delete Domain – removes the SELECTED domain
9	Load Example – fills a 3-domain demo (15 / 5 / 1.667 km)
10	Validate now – runs the checks without generating files

> **CAUTION** **Delete Domain** (8) removes the domain whose radio button is ticked in the extents table. Use **Undo** (5) straight away if you removed the wrong one.

> **TIP** Click **Save Domain…** (4) from time to time. It downloads a `.json` project file that you can re-open later with **Open…**.

STEP 3 Choose projection, domain count and timing

1. Choose the **Map projection** (1): Lambert Conformal, Mercator, Polar Stereographic or Latitude-Longitude.
2. Choose the **Number of domains (max_dom)** (2). The table in Step 4 is rebuilt, but values you already typed are kept.

3. Type the **start** (3) and **end** (4) of the simulation in the format `YYYY-MM-DD_HH:MM:SS`, e.g. `2026-08-01_00:00:00`.
4. Enter the **met input interval** in hours (5), the **WRF time step** in seconds (6) and the **history output** interval in minutes (7).

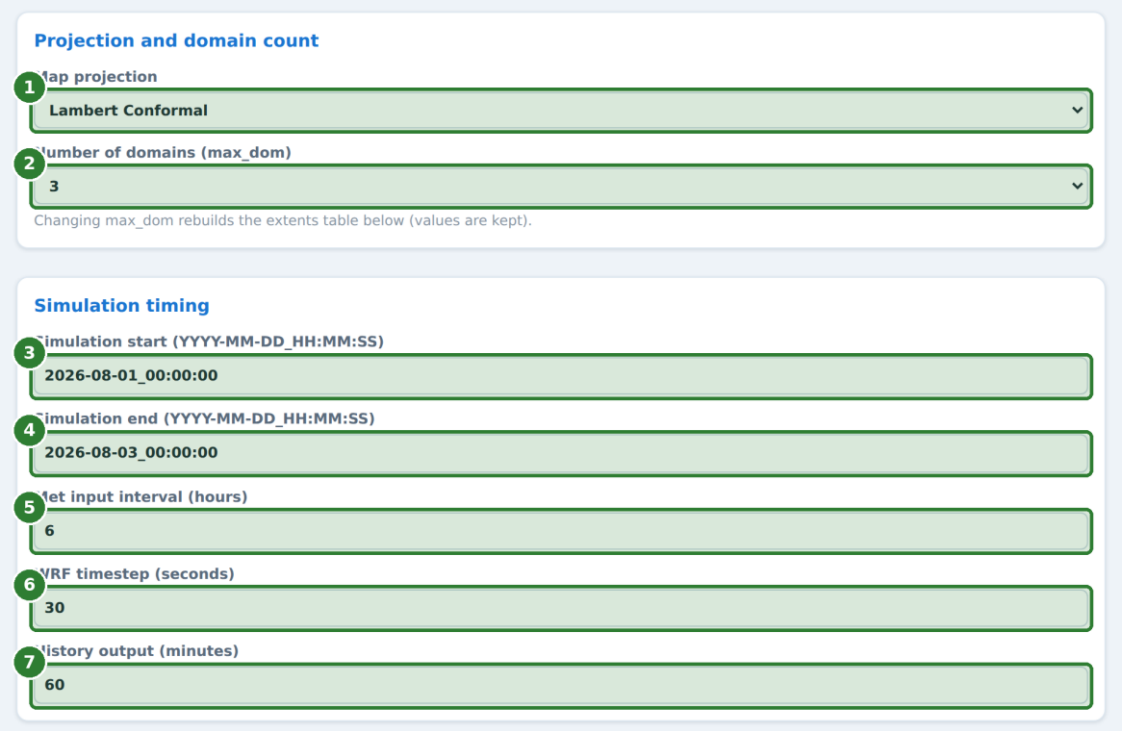


Figure 3. Projection, domain count and simulation timing

1	Map projection – choose the WPS projection (default: Lambert Conformal)
2	Number of domains (max_dom) – 1 to 9; 1 = no nesting
3	Simulation start, format YYYY-MM-DD_HH:MM:SS
4	Simulation end, same format; must be later than the start
5	Interval of your meteorological input data in hours (e.g. 6 for GFS 0.25°/ERA5 6-hourly)
6	WRF time step in seconds (rule of thumb: ≈ 6 × coarsest dx in km)
7	History output frequency in minutes

IMPORTANT The end time must be later than the start time. The interval (5) becomes `interval_seconds` in `namelist.wps` – the example value 6 gives 21600.

STEP 4 Enter domain extents and resolution

Scroll to the card **Domain extents, centre and resolution**. There is one row per domain (d01 is the outermost).

1. For each domain type **Min Lat, Max Lat, Min Lon, Max Lon** in decimal degrees (2). Southern and western values are negative.
2. Type the **Resolution (km)** (4). Nests normally use a 3 : 1 ratio to their parent – the example uses 15 → 5 → 1.667 km.
3. Look at the grey **Cen Lat / Cen Lon** cells (3). They are calculated automatically; you cannot edit them.
4. Tick the **radio button** (1) of a domain to select it. The row is highlighted and the same domain is selected on the map.

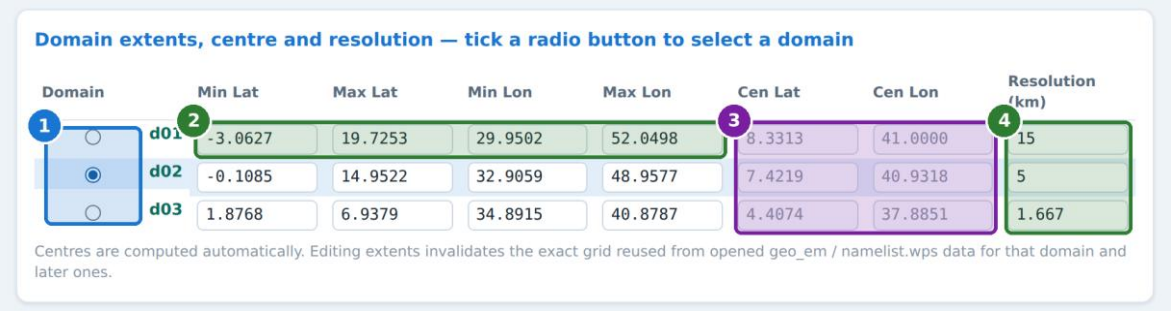


Figure 4. Domain table with d02 selected

1	Radio buttons – tick one to SELECT a domain (it is highlighted here and on the map)
2	Extents of d01: Min Lat, Max Lat, Min Lon, Max Lon (decimal degrees; south/west negative)
3	Cen Lat / Cen Lon – grey, calculated automatically (read-only)
4	Resolution (km) per domain – nests normally use a 3:1 ratio (15 → 5 → 1.667 km)

IMPORTANT A nested domain must lie **inside its parent**. Editing the extents of a domain also discards the exact grid that was reused from an opened `geo_em` / `namelist.wps` for that domain and the ones after it – the grid is recomputed from your new numbers.

TIP You can also define extents graphically (Step 8) and read the resulting numbers back here.

STEP 5 Enter output and geography data paths

1. Type the **Output directory** (1) and the **WPS geog_data_path** (2) – the folder that holds `WPS_GEOG`.
2. Type the **Geogrid** (3) and **Metgrid** (5) output directories.
3. Type the **Ungrib FILE prefix** (4), e.g. `~/Build_WRF/wrf_output/ungrib/FILE`.
4. Optionally use **Browse** (6) to pick a folder or file.

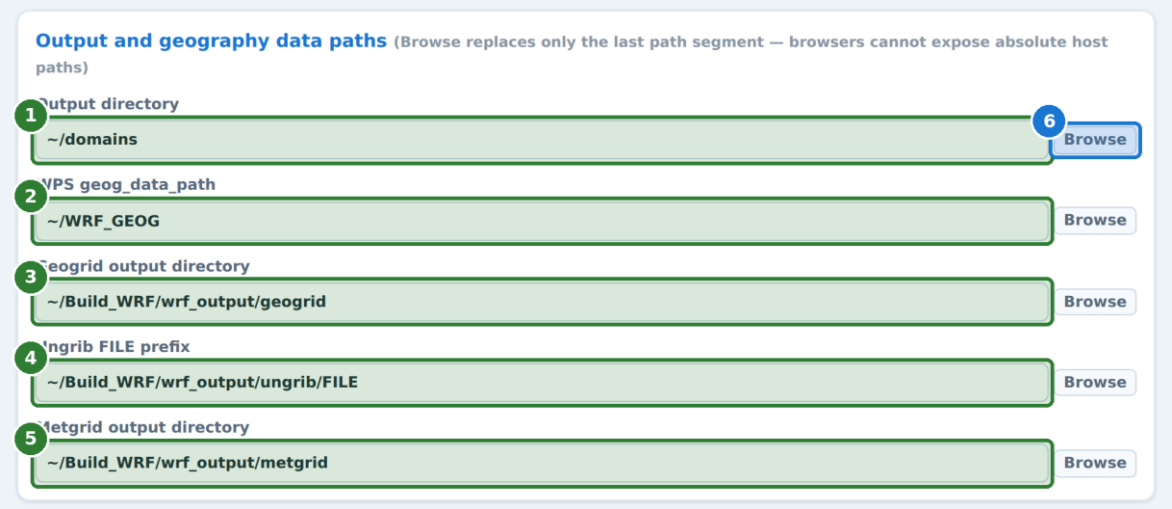


Figure 5. Output and geography data paths

1	Output directory – where you will keep the run files
2	WPS geog_data_path – folder with the WPS_GEOG static data
3	Geogrid output directory (opt_output_from_geogrid_path)
4	Ungrib FILE prefix, e.g. …/ungrib/FILE (prefix = path + file stem)
5	Metgrid output directory (opt_output_from_metgrid_path)
6	Browse – replaces only the LAST path segment (a browser cannot reveal full host paths); type the full path by hand if needed

CAUTION A web browser cannot reveal full folder paths. **Browse** replaces only the **last path segment** of what is in the box. Always check the complete path and type it by hand if necessary.

STEP 6 Switch on optional model components

Plain WRF-ARW needs only `namelist.wps` and `namelist.input` Tick the components you need in the card **Optional model components**.

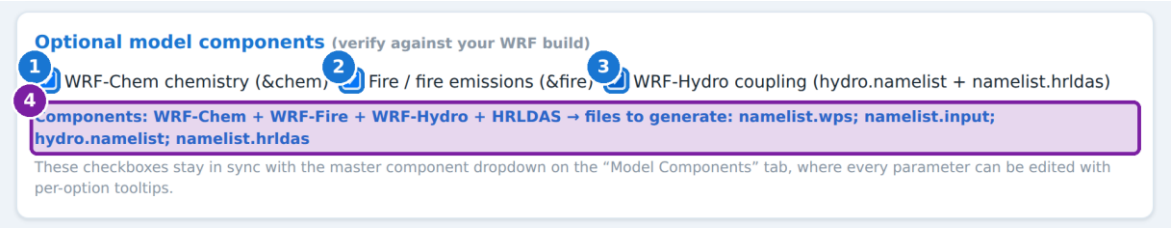


Figure 6. Component tick-boxes and the live summary

1	Tick for WRF-Chem – adds the &chem block to namelist.input
2	Tick for WRF-Fire – adds &fire and sr_x / sr_y
3	Tick for WRF-Hydro – adds hydro.namelist and namelist.hrldas
4	Live summary: which files will be generated

1. Tick **WRF-Chem** (1) to add `&chem`, **Fire** (2) to add `&fire`, and/or **WRF-Hydro** (3) to add `hydro.namelist` and `namelist.hrldas`.
2. Read the summary line (4). With all three ticked it reads "WRF-Chem + WRF-Fire + WRF-Hydro + HRLDAS → files to generate: namelist.wps; namelist.input; hydro.namelist; namelist.hrldas".
3. Optionally open the **Model Components & Namelists** tab. Its **Model components** drop-down is a second way to make the same choice.

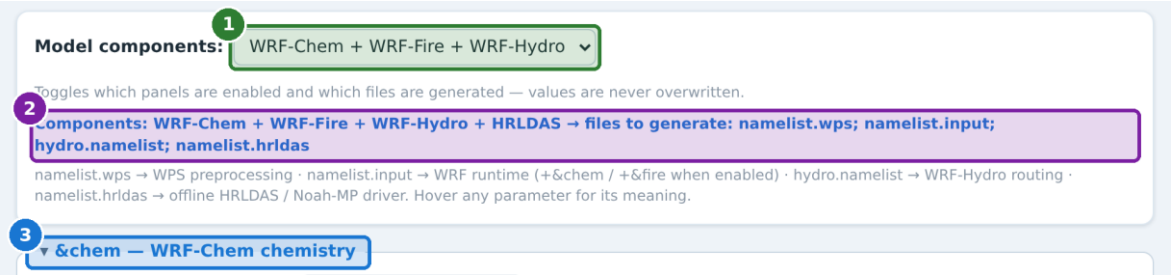


Figure 7. The master drop-down on tab 2

1	Master dropdown – pick the model combination (ARW / Chem / Fire / Hydro and mixes)
2	Summary of the files that will be generated
3	Click a panel title to fold / unfold it; greyed-out panels are switched off

> **NOTE** The drop-down offers seven combinations: WRF-ARW, WRF-Chem, WRF-Fire, WRF-Chem + Fire, WRF-Hydro, WRF-Chem + WRF-Hydro and WRF-Chem + WRF-Fire + WRF-Hydro. Switching components on or off **never overwrites** values you have typed. If the tick-boxes and the drop-down ever look different, trust the summary line – it lists the files that will actually be generated.

STEP 7 Edit the component parameters

Work down the **Model Components & Namelists** tab. Panels for components you did not enable are greyed out. Click a panel title to fold or unfold it.

7.1 WRF-Chem – the &chem panel

Every row shows the parameter name, an editable value and a short description. **Hover over a parameter name** to read the full tooltip.

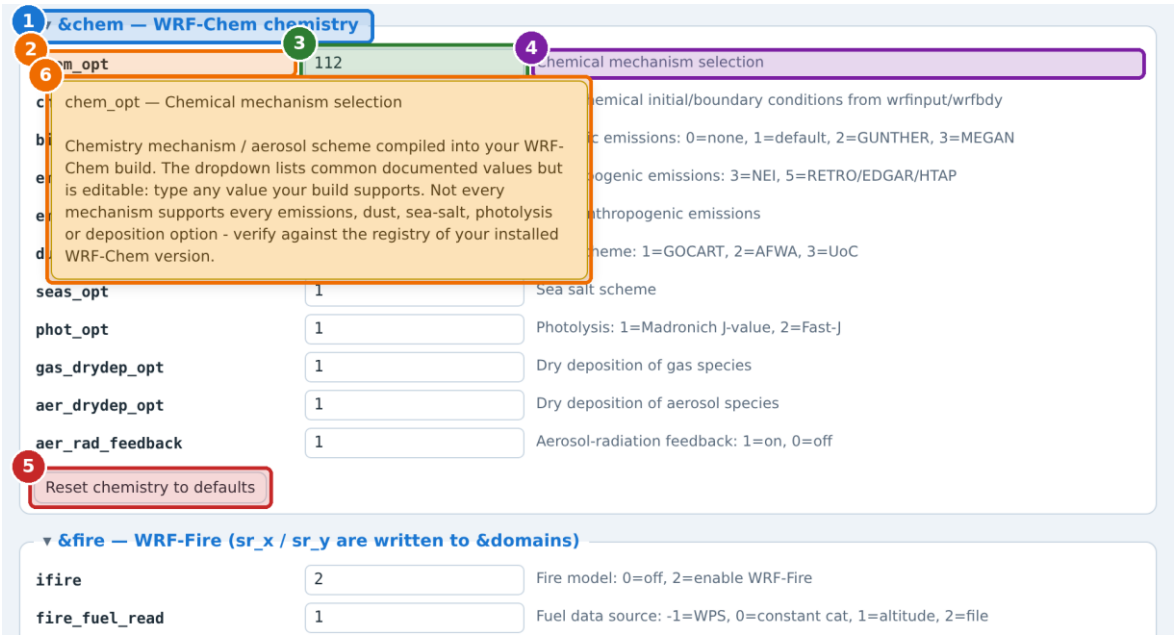


Figure 8. The &chem panel with a tooltip

1	Panel title – click to fold / unfold
2	Parameter name – hover the mouse here to read its full meaning (yellow tooltip)
3	Value – type or overwrite (chem_opt = 112 in this example)
4	Short description of the option
5	Reset chemistry to defaults – discards your edits in this panel
6	Tooltip text appears next to the cursor

IMPORTANT Chemistry options must match the mechanism compiled into **your** WRF-Chem build. Namelist generation does not supply emissions data or chemical initial/boundary files.

7.2 WRF-Fire – the &fire panel

1. Set **ifire** (1) to 2 to enable WRF-Fire, then the fuel and wind settings.
2. Set **sr_x / sr_y** (2). They are written to `&domains`, not `&fire`.
3. Use **+ Add ignition** (4) and **− Remove last ignition** (5) so that the number of cards matches **fire_num_ignitions** (3). The program keeps them equal. Maximum: 5.
4. Fill each ignition card (6): `start_x`, `end_x`, `start_y`, `end_y`, `start_time`, `end_time`, `radius`, `ros`.

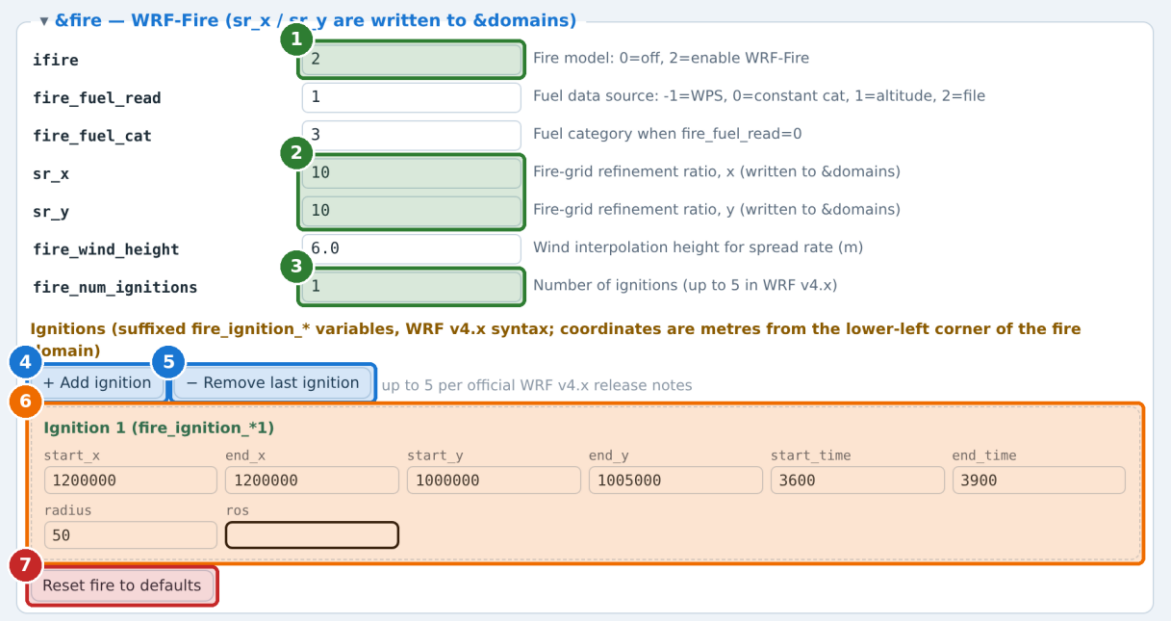


Figure 9. The &fire panel with one ignition

1	ifire – 2 enables the WRF-Fire model
2	sr_x / sr_y – fire-grid refinement ratios (written to &domains)
3	fire_num_ignitions – kept equal to the number of ignition cards
4	+ Add ignition (maximum 5)
5	− Remove last ignition
6	Ignition card: coordinates are METRES from the lower-left corner of the fire domain – NOT longitude/latitude; times are seconds
7	Reset fire to defaults

> **CAUTION** Ignition coordinates are **metres from the lower-left corner of the fire domain** (WRF v4.x syntax). Values that look like longitude/latitude are the classic mistake – the program warns, e.g. "start_x = -108.25 lies outside 0..2445000 m". Some builds use `fire_ignition_start_long/lat` instead; check your release.

7.3 WRF-Hydro – hydro.namelist

1. Choose the **Execution mode** (4): Standalone HRLDAS / Noah-MP sets `sys_cpl = 1`; WRF-coupled Hydro sets `sys_cpl = 2`.
2. Replace each grey `/path/to/…` placeholder in the file fields (2) with the real file – land-model grid, high-resolution routing grid, hydrological table, spatial metadata, optional restart file. Use **Browse** (3) if helpful.
3. Leave the extended routing options (5) blank unless you know your release supports them; empty fields are omitted.

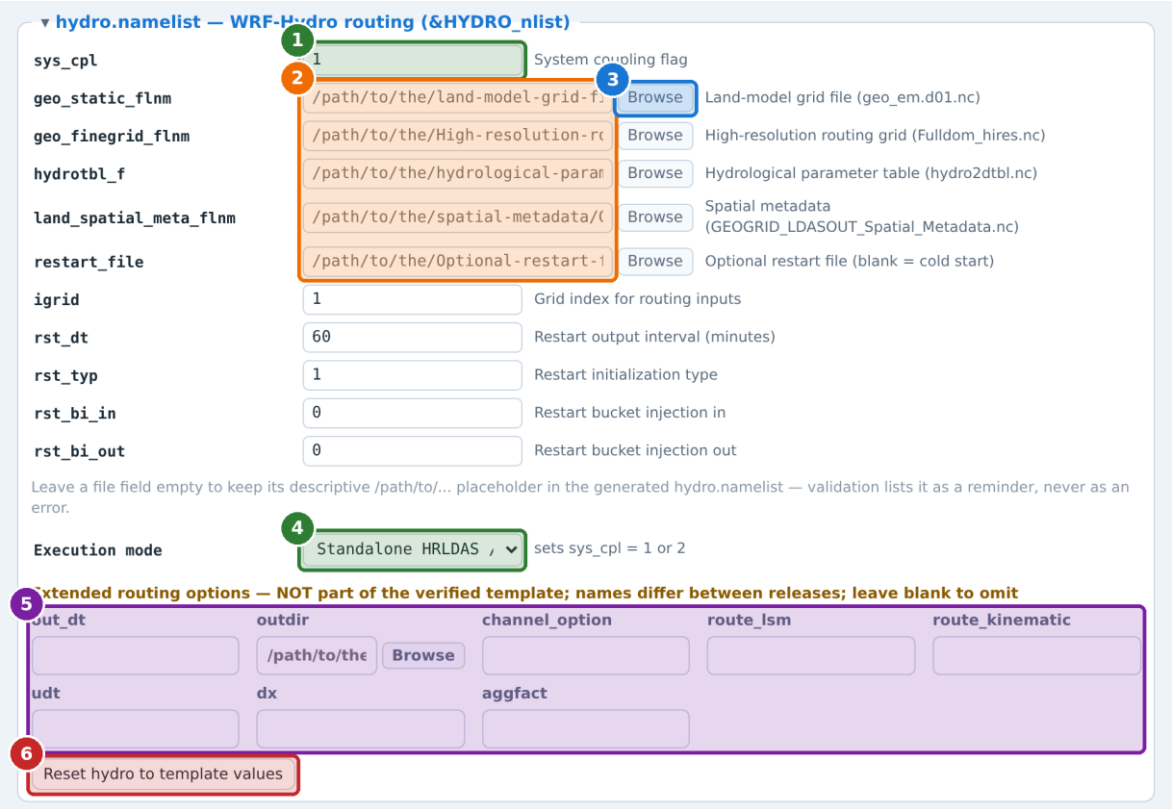


Figure 10. The hydro.namelist panel

1	sys_cpl – set automatically by the Execution mode below
2	File fields – replace every grey "/path/to/…" placeholder with your real file path
3	Browse for a file

4	Execution mode: Standalone HRLDAS/Noah-MP (sys_cpl=1) or WRF-coupled (sys_cpl=2)
5	Extended routing options – optional; leave blank to omit them from the file
6	Reset hydro to template values

IMPORTANT Leaving a placeholder is allowed – the validation panel lists it as a reminder, never as an error – but the model will not run until it is replaced. The &HYDRO_nlist structure follows a supplied template; compare it with the `hydro.namelist` that ships with your build.

7.4 HRLDAS / Noah-MP – namelist.hrldas

1. Fill the paths (1): `HRLDAS_SETUP_FILE`, `INDIR`, `SPATIAL_FILENAME`, `OUTDIR`.
2. Keep **Take HRLDAS start date and KHOUR from the simulation timing** (2) ticked to reuse the timing from tab 1, or untick it to edit the start values (3).
3. Adjust the Noah-MP options (4) and make the forcing variable names (5) match the variables in your `INDIR` files.

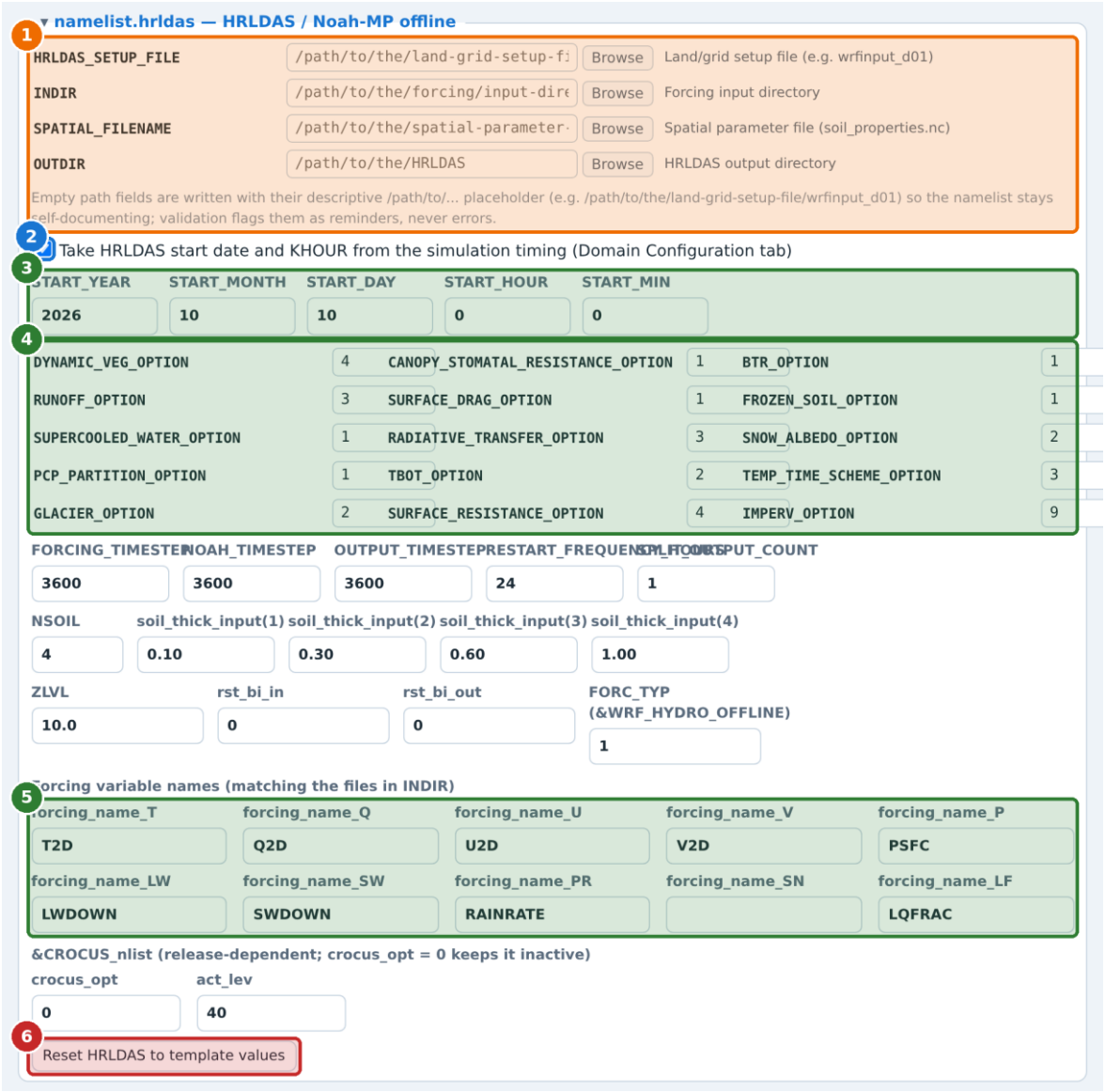


Figure 11. The namelist.hrldas panel

1	Input / output paths (HRLDAS_SETUP_FILE, INDIR, OUTDIR…) – replace the placeholders
2	Tick = take START date and KHOUR from the Simulation timing on tab 1
3	Start year / month / day / hour (editable when sync is OFF)
4	Noah-MP physics options (dynamic_veg_option, runoff_option …)
5	Forcing variable names – must match the variables in your INDIR files
6	Reset HRLDAS to template values

> **NOTE** The `&CROCUS_nlist` group only exists in builds with CROCUS support; `crocus_opt = 0` keeps it inactive.

7.5 Extra physics and generate buttons

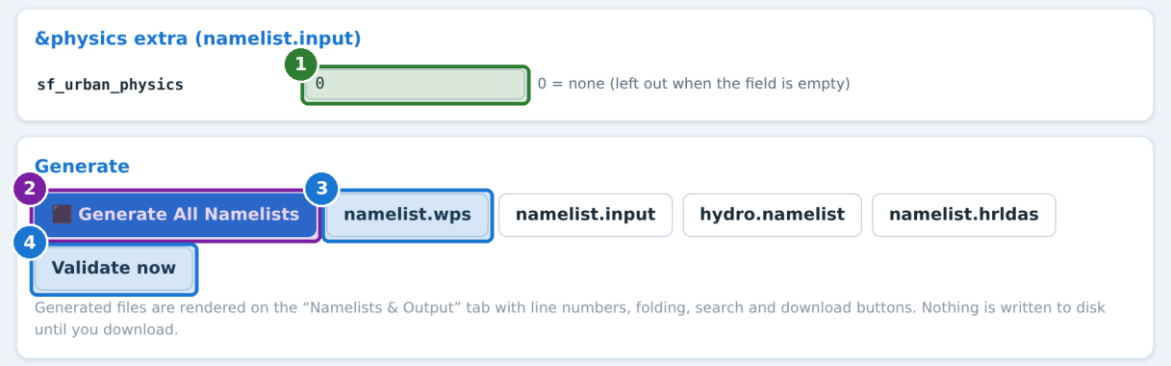


Figure 12. sf_urban_physics and the Generate card

1	sf_urban_physics – optional; 0 = none; empty = omitted from namelist.input
2	Generate All Namelists – the main button
3	Generate / refresh a single file only
4	Validate now

Leave **sf_urban_physics** (1) empty to omit it from `namelist.input`, or enter a value (0 = none). Click **Generate All Namelists** (2) when you are ready – or build a single file with the buttons at (3).

STEP 8 Use the Interactive Domain Wizard

The **Interactive Domain Wizard** tab draws all domains on a map in the chosen projection. Changes made here update the extents table automatically.

8.1 The toolbar

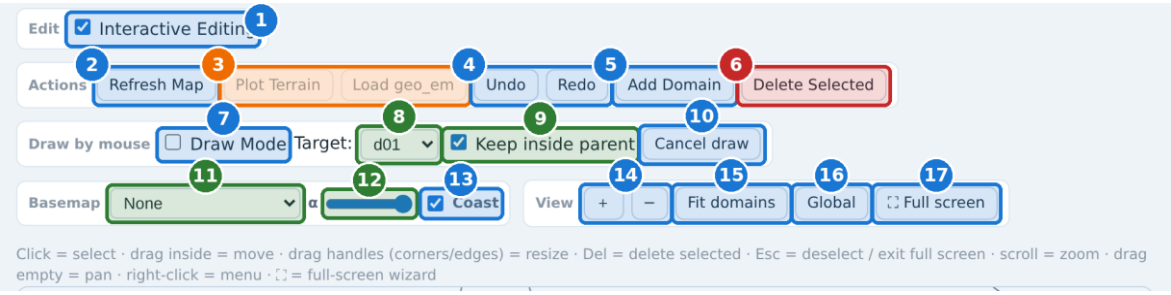


Figure 13. Wizard toolbar

1	Interactive Editing – ON lets you select, move and resize domains with the mouse

2	Refresh Map – redraw the map
3	Plot Terrain and Load geo_em are greyed out: desktop-only features (they need local NetCDF files)
4	Undo / Redo
5	Add Domain
6	Delete Selected – removes the highlighted domain
7	Draw Mode – then drag a rectangle on the map to define a domain
8	Target – which domain the drawn rectangle will define
9	Keep inside parent – forces a nest to stay inside its parent domain
10	Cancel draw
11	Basemap – None, ESRI World Imagery, ESRI World Topo, OpenTopoMap (tiles need internet)
12	Basemap opacity slider (α)
13	Coast – switch the coastline overlay on / off
14	Zoom in (+) / zoom out (−)
15	Fit domains – zoom so all domains are visible
16	Global – show the whole world
17	Full screen (press Esc to leave)

8.2 Select, move and resize

1. Make sure **Interactive Editing** is ticked.
2. **Click** inside a domain to select it. White handles appear on its corners and edges (1, 3).
3. **Drag inside** the domain to move it, or **drag a handle** to resize it.
4. Check the info line at the bottom (5): cursor position, map centre, view size and the selected domain.

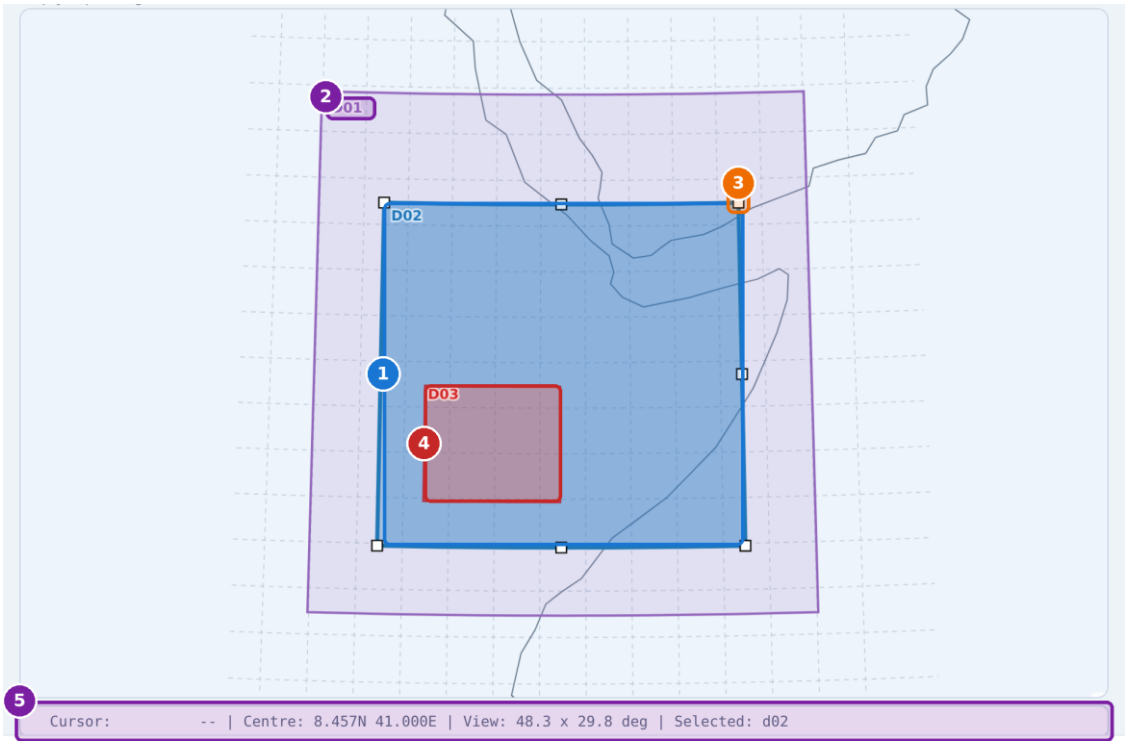


Figure 14. D02 selected on the map

1	Selected domain (D02): click inside any domain to select it – white square handles appear on its corners and edges
2	Domain labels and colours: D01 (outer), D02, D03 (inner nests)
3	Drag a white handle to RESIZE the domain (drag inside the domain to MOVE it)
4	D03 – the innermost nest; it can only be moved inside D02 when ‘Keep inside parent’ is ticked
5	Info line: cursor lat/lon, map centre, view size and the selected domain

Mouse and keyboard summary

Action	Result
Click	Select a domain
Drag inside a domain	Move it
Drag a handle	Resize it
Del	Delete the selected domain
Esc	Deselect / exit full screen
Scroll wheel	Zoom

Action	Result
Drag empty map	Pan
Right-click on a domain	Context menu

8.3 Right-click menu

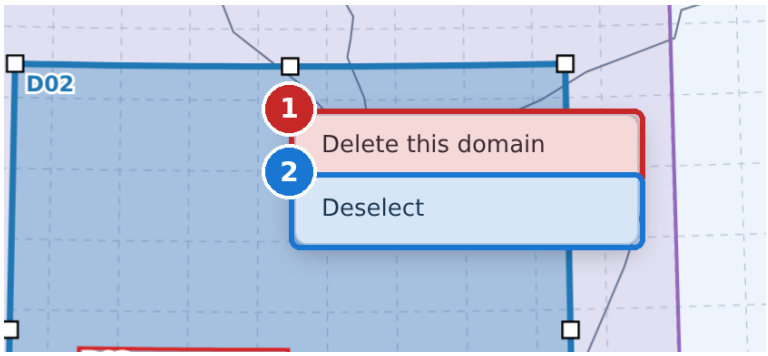


Figure 15. Context menu

1	Delete this domain
2	Deselect

8.4 Draw a domain with the mouse

1. Tick **Draw Mode** (1).
2. Pick the **Target** domain (2), e.g. d03.
3. Keep **Keep inside parent** (3) ticked for nests.
4. Press the mouse button on the map, drag a rectangle (4) and release. The target domain now has those limits.
5. If you are unhappy, press **Undo** or **Cancel draw**.

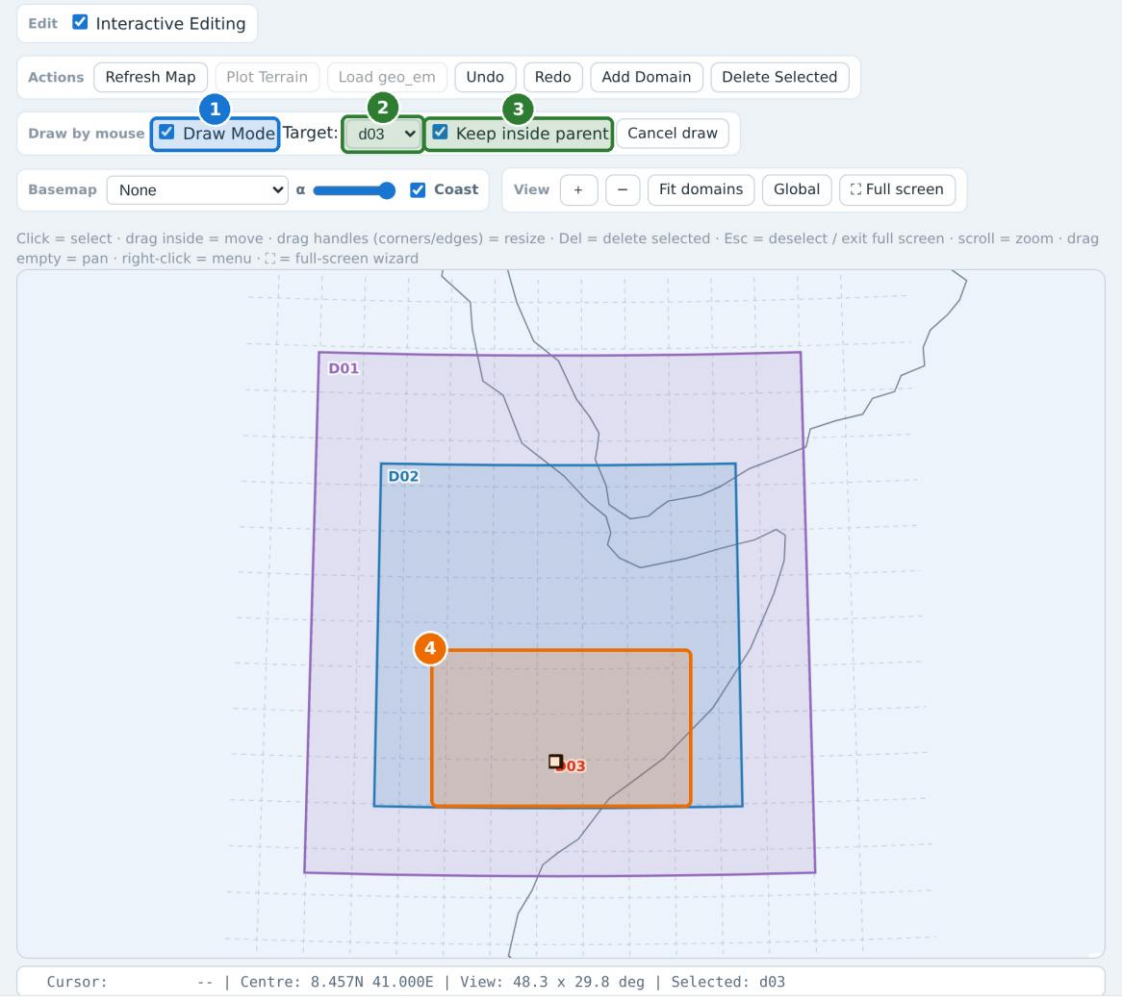


Figure 16. Drawing d03 inside d02

1	Tick Draw Mode
2	Choose the Target domain (here d03)
3	Leave ‘Keep inside parent’ ticked for nests
4	Press, drag and release on the map: the rectangle becomes the new extent of the target domain

TIP Basemap: choose ESRI World Imagery, ESRI World Topo or OpenTopoMap to see terrain under your domains (needs internet), and use the α slider for transparency. **Fit domains** zooms to all domains; ⛶ **Full screen** enlarges the map (Esc leaves).

STEP 9 Generate and validate the namelists

1. Click **Generate namelists** on tab 1, or **Generate All Namelists** on tab 2 or 4. Use **Validate now** to run only the checks.

2. If the program has advice for you, a **“Warnings before generation”** window opens. Read it.
3. Click **Yes** to continue, or **No** to go back and fix your settings.

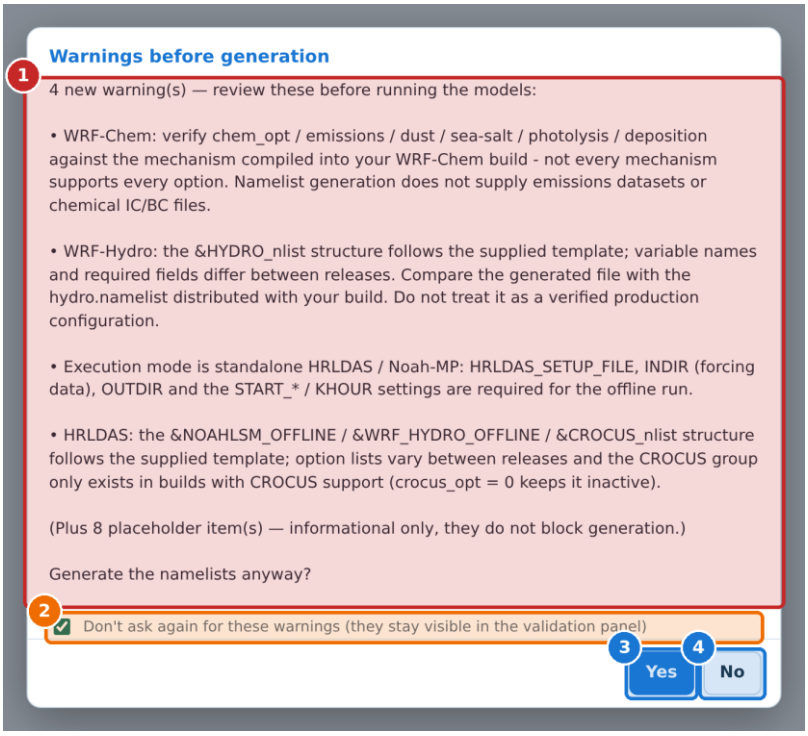


Figure 17. Warnings before generation

1	Read each warning – they are advice, not errors; generation is still possible
2	‘Don’t ask again’ – warnings stay visible in the validation panel
3	Yes – generate the files
4	No – go back and fix your settings

> **NOTE** Warnings are advice, not errors. The window above lists warnings such as: verify `chem_opt` against your build, ignition coordinates outside the domain range, and the template status of the Hydro and HRLDAS blocks. Ticking **Don’t ask again** only hides this window – the warnings stay in the validation panel.

After **Yes** the **Namelists & Output** tab shows the result:

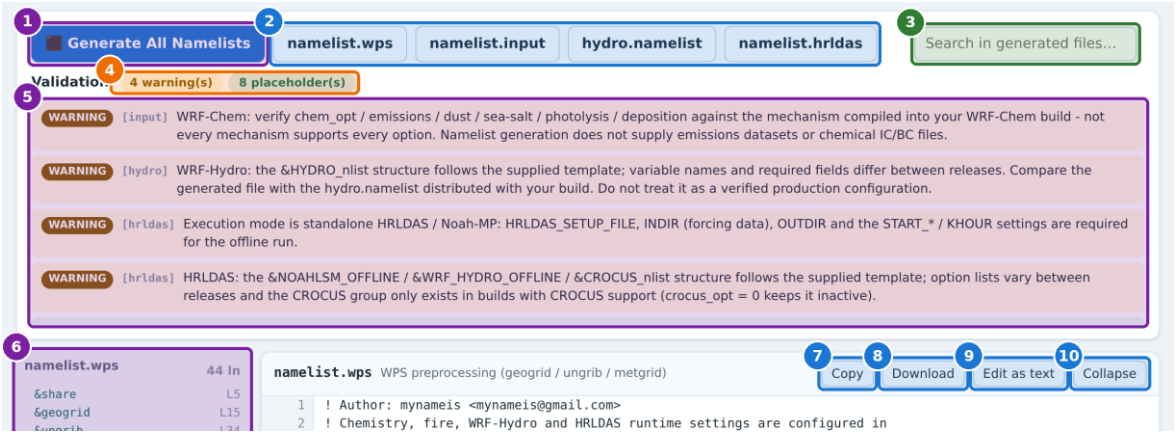


Figure 18. Validation panel and generated files

1	Generate All Namelists (re-run after any change)
2	Individual file buttons: namelist.wps, namelist.input, hydro.namelist, namelist.hrldas
3	Search box – matches are highlighted in yellow across all files
4	Counters: warnings and placeholders still to be replaced
5	Validation list: errors (red), warnings (amber) and placeholders – scroll inside the box to read all items
6	Outline – click a section name (&share, &domains, &fire …) to jump to it
7	Copy – copies the file text to the clipboard
8	Download – saves the file to your computer
9	Edit as text – hand-edit the generated text
10	Collapse / expand the file view

1. Read the counters (4) – e.g. “4 warning(s)” and “8 placeholder(s)”. Scroll the validation list (5) to read every item.
2. Use the **outline** (6) on the left to jump to `&share`, `&geogrid`, `&domains`, `&chem`, `&fire` and so on.
3. Check the status bar for “SUCCESS — rendered namelist.wps, namelist.input, hydro.namelist, namelist.hrldas below”.

Search inside the files

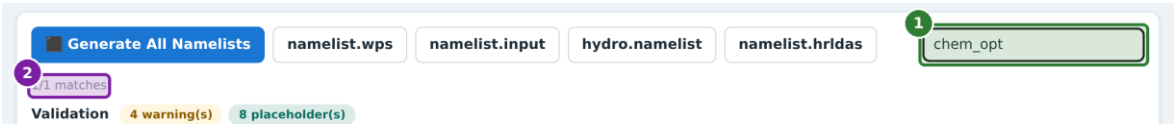


Figure 19. Searching for chem_opt

1	Type a parameter name, e.g. chem_opt, then press Enter to step through matches
2	Match counter (1/2 matches …)

Type a name in the search box (1), press **Enter**, and step through the matches. The counter (2) shows e.g. "1/2 matches"; matches are highlighted in yellow.

> **IMPORTANT** Resolve **placeholders** before running the model: they are the grey `/path/to/…` values you have not replaced. **Errors** (shown in red) must be fixed – the namelist would be invalid.

STEP 10 Download the files and run WRF

1. On each file card click **Download** (8 in Figure 18) to save it. **Copy** puts the text on the clipboard; **Edit as text** lets you make a last manual change; **Collapse** hides the file view.
2. Click **Save Domain…** on tab 1 to keep a `.json` project so you can regenerate the files later.
3. Copy `namelist.wps` to your WPS directory, and `namelist.input` (plus `hydro.namelist` / `namelist.hrldas` if used) to your WRF run directory.
4. Run the usual WRF chain: `geogrid.exe`, `ungrib.exe`, `metgrid.exe`, then `real.exe` and `wrf.exe`.

> **CAUTION** Download your files **before** closing the browser tab. The program does not write anything to disk by itself.

Dark theme

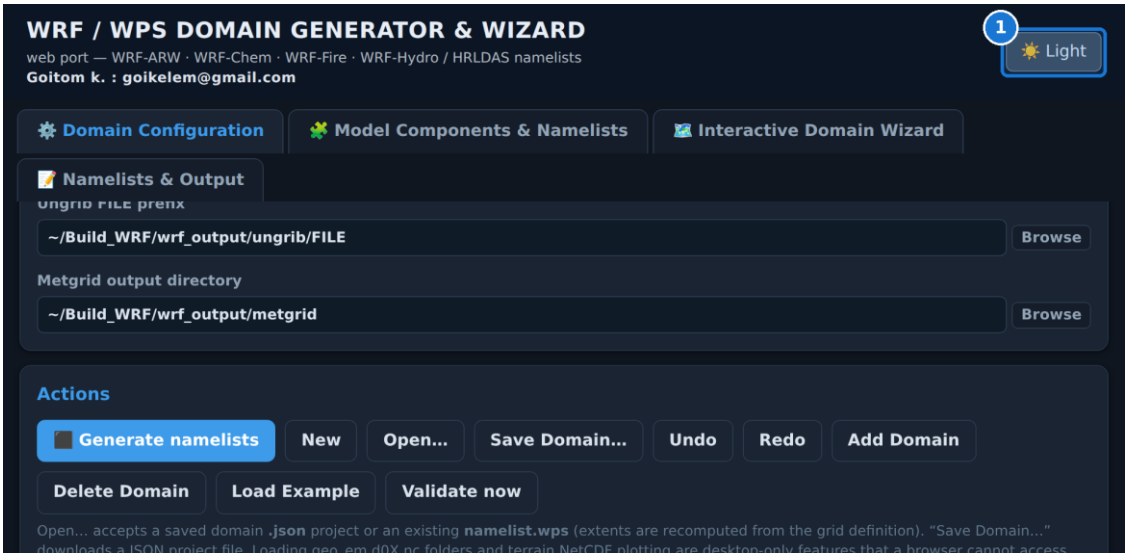


Figure 20. Dark theme

1	Click again to return to the light theme

Click the theme button (1) in the header to switch between light and dark. The same button switches back.

Appendix A Worked example

Click **Load Example** and you get these values. They produce the `namelist.wps` entries shown on the right-hand side.

Domain	Min Lat	Max Lat	Min Lon	Max Lon	Resolution (km)
d01	-3.0627	19.7253	29.9502	52.0498	15
d02	-0.1085	14.9522	32.9059	48.9577	5
d03	1.8768	6.9379	34.8915	40.8787	1.667

Setting	Value
Projection	Lambert Conformal
Simulation	2026-08-01_00:00:00 → 2026-08-03_00:00:00
Met interval / time step / history	6 h / 30 s / 60 min
`parent_id`	1, 1, 2
`parent_grid_ratio`	1, 3, 3
`i_parent_start`, `j_parent_start`	1, 24, 45 (both)
`e_we`	164, 358, 400
`e_sn`	172, 337, 340
`dx`, `dy`	15000
`map_proj`	lambert
`ref_lat`, `ref_lon`	8.33130, 41.00000

Appendix B Parameter reference

Default values as shown in the program. Hover over a parameter name in the application for the full tooltip.

&chem

Parameter	Default	Meaning
chem_opt	112	Chemical mechanism selection
chem_in_opt	1	Read chemical initial/boundary conditions from wrfinput/wrfbdy
bio_emiss_opt	3	Biogenic emissions: 0=none, 1=default, 2=GUNTHER, 3=MEGAN
emiss_opt	5	Anthropogenic emissions: 3=NEI, 5=RETRO/EDGAR/HTAP
emiss_inpt_opt	1	Read anthropogenic emissions
dust_opt	1	Dust scheme: 1=GOCART, 2=AFWA, 3=UoC
seas_opt	1	Sea salt scheme
phot_opt	1	Photolysis: 1=Madronich J-value, 2=Fast-J
gas_drydep_opt	1	Dry deposition of gas species
aer_drydep_opt	1	Dry deposition of aerosol species
aer_rad_feedback	1	Aerosol-radiation feedback: 1=on, 0=off

&fire

Parameter	Default	Meaning
ifire	2	Fire model: 0=off, 2=enable WRF-Fire
fire_fuel_read	1	Fuel data source: -1=WPS, 0=constant cat, 1=altitude, 2=file
fire_fuel_cat	3	Fuel category when fire_fuel_read=0
sr_x	10	Fire-grid refinement ratio, x (written to &domains)
sr_y	10	Fire-grid refinement ratio, y (written to &domains)
fire_wind_height	6.0	Wind interpolation height for spread rate (m)
fire_num_ignitions	1	Number of ignitions (up to 5 in WRF v4.x)

hydro.namelist (&HYDRO_nlist)

Parameter	Default	Meaning
sys_cpl	1	System coupling flag
geo_static_flnm	(empty)	Land-model grid file (geo_em.d01.nc)
geo_finegrid_flnm	(empty)	High-resolution routing grid (Fulldom_hires.nc)
hydrotbl_f	(empty)	Hydrological parameter table (hydro2dtbl.nc)
land_spatial_meta_flnm	(empty)	Spatial metadata (GEOGRID_LDASOUT_Spatial_Metadata.nc)
restart_file	(empty)	Optional restart file (blank = cold start)
igrid	1	Grid index for routing inputs
rst_dt	60	Restart output interval (minutes)
rst_typ	1	Restart initialization type
rst_bi_in	0	Restart bucket injection in
rst_bi_out	0	Restart bucket injection out

namelist.hrldas – paths

Parameter	Default	Meaning
HRLDAS_SETUP_FILE	(empty)	Land/grid setup file (e.g. wrfinput_d01)
INDIR	(empty)	Forcing input directory
SPATIAL_FILENAME	(empty)	Spatial parameter file (soil_properties.nc)
OUTDIR	(empty)	HRLDAS output directory

Appendix C Troubleshooting

Problem	Likely cause	What to do
Plot Terrain / **Load geo_em** are greyed out	Desktop-only features; browsers cannot read local NetCDF files	Use **Open…** with a `.json` project or a `namelist.wps`.

Problem	Likely cause	What to do
Map shows no background	Basemap tiles need internet, or Basemap is “None”	Select a basemap and check your connection; “None” still shows domains and coast.
Browse gives only part of a path	Browser security	Type the full path in the box.
Warning: ignition coordinate outside range	Longitude/latitude typed instead of metres	Enter metres from the lower-left corner of the fire domain.
Many “placeholder” items	Grey `/path/to/…` entries not replaced	Fill the Hydro and HRLDAS file fields.
Nest cannot leave its parent while drawing	**Keep inside parent** is ticked	Keep it ticked (recommended) or enlarge the parent first.
Grid changed after editing extents	Exact grid from an opened file is discarded when extents change	Expected – check the new grid in the output.
Work lost after closing the page	Nothing is saved automatically	Use **Save Domain…** and **Download**.
Drop-down and tick-boxes differ	Two controls for one setting	Read the summary line; set one control again.

> **NOTE** Generated namelists are starting templates. Always compare them with the sample namelists of your own WRF, WPS and WRF-Hydro releases before running production simulations.

Appendix G. WRFall User Guide: Downloading Data and Running WRF

Editorial note (reorganization): the quick guide "WRFall User Guide: Downloading Data and Running WRF", which was embedded in the source file between Chapters 10 and 11, is reproduced verbatim in this appendix so the numbered chapters run continuously.

Based on wrfall_install.pl v1.4.2 and your GFS example for the ethiowrf domain (16 October 2026, 00 UTC, 18-hour forecast).

1. What WRFall does

wrfall is one command that handles the daily WRF workflow. This guide covers the two daily steps:

1. **--getdata** downloads GFS, ECMWF IFS or ERA5 data for your domain only.
2. **--run** runs the whole chain: ungrib.exe, metgrid.exe, real.exe and wrf.exe. geogrid.exe is optional.

Both steps assume WRF/WPS is already installed (--install -arw) and a domain exists (--mkdomain ethiowrf).

2. Before you start

2.1 Work from inside your domain

A domain created with --mkdomain contains a wrfall link to the main script, so you can work from inside it:

```
cd                                                    ~/Build_WRF/ethiowrf
./wrfall --help
```

If you are not inside the domain, add --domain ~/Build_WRF/ethiowrf to every command.

2.2 Domain folder layout

Item	Purpose
namelist.wps, namelist.input	Your domain and model settings. Edit these first.
GFS_Input/, ECMWF_Input/, ERA5_Input/	Downloaded data, one sub-folder per cycle (YYYYMMDDCC).
wrf_output/geogrid, ungrib, metgrid, Plots	Archived products of each step.

Item	Purpose
wrf_output/YYYYMMDDCC/	Final wrfout_* files of each run.
domain_env.sh	Environment variables for the domain (source domain_env.sh).
wrfall	Link to the main script.

2.3 Check namelist.wps first

The download area is read from namelist.wps (domain d01), so set your domain there first (for example with WRF-domain-Wizard). Make sure geo_em.d01.nc (and geo_em.d02.nc if you use nesting) already exist, because the run skips geogrid by default.

3. Downloading data (--getdata)

3.1 Basic command

```
./wrfall --getdata gfs --date 20261016 --end 18
```

This downloads the GFS 0.25° forecast for 16 October 2026, 00 UTC cycle, from forecast hour 0 to 18, every 3 hours.

3.2 What you should see

```
Welcome to WRFall. You are downloading GFS data.

*  Domain extent (namelist.wps, d01, mercator): lat -4.58 to 23.21, lon
24.61                              to                              55.99
   Download area (+1.00 deg margin): lat -5.75 to 24.25, lon 23.50 to
57.00

*         GFS     20261016     00Z     f000-f018     every     3h     ->
.../ethiowrf/GFS_Input/2026101600
   [get                   ]                   gfs.t00z.pgrb2.0p25.f000
   ...
*  Done. Failed files: 0
```

Reading the output:

- The **domain extent** is calculated from your namelist.wps.
- The **download area** is that extent plus a margin (default 1.0°), rounded outward to the 0.25° grid. Only this region is downloaded, so files stay small.
- Seven files (f000, f003, ..., f018) are saved in GFS_Input/2026101600/. Each filename starts with the date and cycle, for example 2026101600_gfs.t00z.pgrb2.0p25.f000.

- Failed files: 0 means every file was downloaded.

3.3 Options

Option	Meaning	Default
--getdata gfs|ecmwf|era5	Data source	required
--date YYYYMMDD	Forecast start date (UTC)	today (UTC)
--cycle 00|06|12|18	Model cycle	00
--start H	First forecast hour	0
--end H	Last forecast hour	120 for GFS/ECMWF, 21 for ERA5
--step H	Hours between files	3
--margin DEG	Extra degrees around the domain	1.0
--bbox W E S N	Set the area by hand instead of reading namelist.wps	none
--domain DIR[,DIR2]	Domain folder(s); several can be given separated by commas	current folder
--outdir DIR	Save somewhere other than the default folder	none
--force	Download again even if the file exists	off
--dryrun	Show what would happen without downloading	off
--wget / --curl	Force a download tool	auto

3.4 Examples

```
#       72-hour       GFS       forecast,       12       UTC       cycle
./wrfall   --getdata   gfs   --date   20261016   --cycle   12   --end   72

#         Larger         margin         around         the         domain
./wrfall   --getdata   gfs   --date   20261016   --end   48   --margin   2

#             ECMWF             IFS             0.25             degree
./wrfall     --getdata     ecmwf     --date     20261016     --end     48

#   ERA5   (needs   the   cdsapi   package   and   a   CDS   account   key)
./wrfall                          --addpack                          cdsapi
./wrfall --getdata era5 --date 20261016 --end 21
```

3.5 Good to know

- **Re-running is safe.** Files that already exist are shown as [skip], and interrupted downloads resume. Use --force to download them again.
- **Match your data to your run.** The forecast hours you download must cover what you run. For --run ... 18 you need data out to f018.
- **GFS availability.** GFS comes from NOAA NOMADS, which keeps only recent cycles (roughly the last 10 days), and a cycle appears online a few hours after its nominal time. A [FAIL] ... data not yet available line means the file is not online yet or the date is too old. Wait or choose another cycle.
- **ERA5** is a reanalysis, so it does not need forecast hours in the same way. WRFall requests surface and pressure-level files for each calendar day covered.

4. Running WRF (--run)

4.1 Basic command

```
./wrfall --run 20261016 00 18
```

The three values are **date**, **cycle** and **forecast hours**. Add gfs, ecmwf or era5 at the end to choose the data source (the default is gfs):

```
./wrfall        --run        20261016        00        18        gfs
./wrfall --run auto gfs
```

--run auto uses today's date, the 00 UTC cycle and a 72-hour forecast. Forecast hours must be **between 1 and 72**. or Edit the script

4.2 What happens, step by step

Step	What WRFall does	Main products
1. Set the time	Backs up both namelists (.bak.<timestamp>), then writes start and end dates into namelist.wps	namelist.wps.bak.*
2. geogrid (optional)	**Skipped by default.** Runs only if you set RUN_GEOGRID=1	geo_em.d01.nc, geo_em.d02.nc
3. Link GRIB and Vtable	Links your downloaded files with link_grib.csh and selects the correct Vtable	GRIBFILE.AAA ..., Vtable, log.link_grib
4. ungrib.exe	Converts GRIB files to intermediate format	log.ungrib, archived in wrf_output/ungrib/
5. metgrid.exe	Interpolates data onto your domain	met_em.d01.*, met_em.d02.*, log.metgrid
6. Update namelist.input	Sets run length and start/end date and time	updated namelist.input
7. real.exe (2 MPI ranks)	Builds initial and boundary conditions	wrfinput_d01, wrfinput_d02, wrfbdy_d01, log.real
8. wrf.exe (8 MPI ranks)	Runs the forecast	wrfout_d01_*, wrfout_d02_*, log.wrf, rsl.out.*, rsl.error.*

When the run finishes, the wrfout_* files are moved to wrf_output/2026101600/ and the working met_em* files are removed from the domain folder. A copy of each stays in wrf_output/metgrid/2026101600/. In the moved files, colons and dashes in the filename become underscores, for example wrfout_d01_2026_10_16_00_00_00.

4.3 What you should see

```
Welcome to WRFall. You are running WRF.

*  WRF run started in ethiowrf. Source=gfs, date=20261016, cycle=00
UTC, forecast=18 hours
*  Skipping geogrid.exe (set RUN_GEOGRID=1 to run it)
*  Linking GRIB files and Vtable
*  Running ungrib.exe
*  Running metgrid.exe
*  Updating namelist.input
*  Running real.exe with 2 MPI rank(s)
*  Running wrf.exe with 8 MPI rank(s)

✓ WRF run completed. Output: .../wrf_output/2026101600 (elapsed ...)
```

4.4 Settings you can change

Set these in the command line before ./wrfall:

Variable	Meaning	Default
RUN_GEOGRID=1	Run geogrid.exe (needed for a new or changed domain)	off
WRF_NP=N	MPI ranks for wrf.exe	8
REAL_NP=N	MPI ranks for real.exe	2

Examples:

```
# first run of a new domain: build geo_em files too
RUN_GEOGRID=1 ./wrfall --run 20261016 00 18

# machine with 16 physical cores
WRF_NP=16 ./wrfall --run 20261016 00 18
```

Use no more ranks than physical cores. Hyper-threading usually does not help WRF.

4.5 Settings that stay in your namelists

WRFall only writes the dates and run length. Everything else comes from your files:

- interval_seconds in namelist.wps must match how often your data is available. If you downloaded 3-hourly files but your namelist says 21600, only 6-hourly met_em files are made, as in your example list (00, 06, 12, 18). That is valid, but use 10800 if you want boundary conditions every 3 hours.
- max_dom, grid sizes, physics and history_interval in namelist.input control the domains and how many wrfout files you get.

5. Complete daily workflow

```
cd                                                  ~/Build_WRF/ethiowrf

#     1.     download     data     (00     UTC     cycle,     18     h)
./wrfall   --getdata   gfs   --date   20261016   --cycle   00   --end   18

#    2.    run    WRF    for    the    same    date,    cycle    and    length
./wrfall            --run            20261016            00            18

#              3.              find              your              results
ls wrf_output/2026101600/
```

The date, cycle and hours in step 2 must match step 1.

6. Checking results

File	Use it to
log.ungrib, log.metgrid, log.real, log.wrf	See each program's messages
rsl.error.0000	Find the **real** cause when real.exe or wrf.exe fails
rsl.out.0000	Check progress and timing; a good run ends with wrf: SUCCESS COMPLETE WRF
wrf_output/YYYYMMDDCC/wrfout_*	Open in Python, NCL, ncview or other tools

For a quick look at a result file: ncdump -h wrfout_d01_2026_10_16_00_00_00 | less.

7. Troubleshooting

Message or problem	Likely cause and fix
Could not read the domain extent from .../namelist.wps	Check namelist.wps, or pass --bbox W E S N.
[FAIL] gfs.t00z...	Data not online yet, or the date is too old for NOMADS. Try later or a different cycle.
GRIB input directory not found	You have not downloaded the data for this date and cycle. Run --getdata with the same date and cycle.
No GRIB files found	The cycle folder is empty. Download again.

Message or problem	Likely cause and fix
Required file not found in the domain	WRF/WPS executables are not linked in the domain. Re-create or refresh the domain (--mkdomain, or --update).
Bad arguments: date YYYYMMDD, cycle 00|06|12|18, hours 1-72	Fix the argument. Forecast hours cannot exceed 72 with --run.
metgrid produced no met_em files	Check log.metgrid. Remove stale or read-only met_em* files and run again.
real.exe failed	Read rsl.error.0000. Common causes: missing forecast hours in your data, or wrong dates in the namelists.
No wrfout_* files found	wrf.exe stopped early. Check rsl.error.0000 (often a CFL or segmentation error: reduce time_step).
geo_em file missing	Run once with RUN_GEOGRID=1, and confirm the geography data (WPS_GEOG) is installed.

8. Quick reference

```
#                    create                    a                    domain
./wrfall                          --mkdomain                          ethiowrf

#                               download                                  data
./wrfall   --getdata   gfs   --date   YYYYMMDD   --cycle   00   --end   72

#                                  run                                    WRF
./wrfall            --run            YYYYMMDD            00            72
./wrfall                  --run                  auto                  gfs

#                              useful                                  extras
./wrfall   --getdata   gfs   --dryrun   ...              #  preview  only
RUN_GEOGRID=1   ./wrfall   --run   ...                 #  include  geogrid
WRF_NP=16  ./wrfall  --run  ...                     #  change  core  count
./wrfall  --help                                          #  short  help
./wrfall --flags                          # all options
```